

Compiladores (CC3001) — Resumo Teórico

Gonçalo Sousa

Atualizado: Semana 2 (Aulas 1–5)

Conteúdo

Aula 1 — Introdução e fases de um compilador	3
O que é um compilador	3
Compiladores vs. interpretadores	3
Fases de um compilador	3
Self-hosting e bootstrapping	5
Aula 2 — Análise lexical	6
O que faz a análise lexical	6
Expressões regulares	6
Autômatos finitos determinísticos (DFA)	8
Autômatos finitos não-determinísticos (NFA)	8
Conversão de NFA em DFA: construção de subconjuntos	10
Minimização de autômatos	12
Aula 3 — Geradores de analisadores lexicais	14
Implementação direta de um DFA em C	14
Alex (gerador para Haskell)	15
Flex (gerador para C)	16
Longest match e first match	17
Comparação rápida Alex vs. Flex	17
Aula 4 — Análise sintática e gramáticas independentes de contexto	18
O que é a análise sintática	18
Gramáticas independentes de contexto	18
Derivações	19
Linguagem descrita por uma gramática	20
Árvores sintáticas	22
Gramáticas ambíguas	23
Exemplo central: expressões aritméticas	25
Eliminar ambiguidade: associatividade e precedência	25
O problema do “dangling else”	27
Aula 5 — Análise sintática <i>bottom-up</i> (LR)	28
Análise <i>bottom-up</i> e analisadores LR	28
Tabela de <i>parsing</i> LR e o algoritmo	29
Autômato e tabela LR(0)	32
Análise SLR(1)	37
Conflitos e como resolvê-los	40
Extras: LR(1) e LALR(1)	40

Aula 1 — Introdução e fases de um compilador

O que é um compilador

Um **compilador** é um tradutor de programas: recebe um programa escrito numa linguagem (a *linguagem fonte*) e produz um programa equivalente noutra linguagem (a *linguagem destino*). Normalmente traduz de uma linguagem de **alto nível** (ex: C, Java) para uma de **baixo nível** (código máquina, ou uma representação intermédia como bytecode JVM). É a principal técnica usada para implementar linguagens de programação — GCC (C/C++ → código máquina), Javac (Java → bytecode JVM), GHC (Haskell → código máquina) e Scalac (Scala → bytecode JVM) são todos compiladores.

Exemplo resolvido

Um caso interessante é **elm**, que compila Elm para **JavaScript** — o destino nem sempre é código máquina; pode ser outra linguagem de alto nível (a isto também se chama *transpilação*, mas o mecanismo interno é o mesmo de um compilador).

Compiladores vs. interpretadores

Em vez de traduzir o programa todo antes de o correr, um **interpretador** faz a análise lexical e sintática da mesma forma que um compilador, mas depois **executa diretamente** o programa a partir da árvore sintática (ou, em alternativa, gera e executa código intermédio à medida que avança, sem produzir um executável final).

Definição — Compilação vs. interpretação

Compilação: traduz o programa fonte inteiro para outra linguagem *antes* de o executar, produzindo um artefacto (executável) reutilizável. **Interpretação:** analisa e executa o programa diretamente, sem produzir um executável separado.

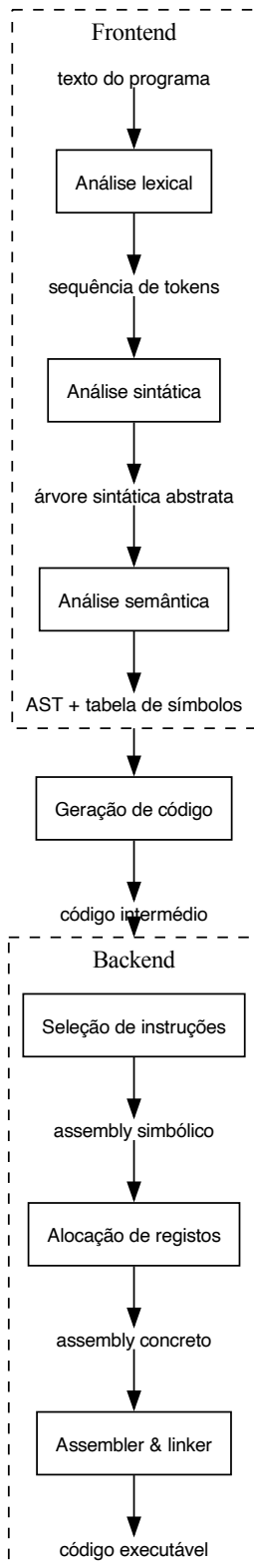
	Vantagem
Interpretação	Mais simples de implementar; mais fácil suportar várias arquiteturas; permite REPL (<i>read-eval-print-loop</i>), útil para desenvolvimento interativo.
Compilação	O código compilado é normalmente mais eficiente; o executável pode ser distribuído sem depender do compilador.

Atenção

Nota adicional: muitas implementações reais **combinam** as duas técnicas (ex: GHC pode compilar ou interpretar Haskell; OCaml tem `ocaml` — interpretador/REPL — e `ocamlopt` — compilador). Não penses nelas como mutuamente exclusivas.

Fases de um compilador

Um compilador divide-se tradicionalmente em duas grandes partes, ligadas por uma **representação intermédia** (código intermédio) que é independente da linguagem fonte e da máquina destino:



Definição — Frontend / Backend

Frontend (depende da linguagem fonte, não da máquina destino): análise lexical → análise sintática → análise semântica → AST + tabela de símbolos.

Backend (depende da máquina destino, não da linguagem fonte): seleção de instruções → alocação de registos → assembler & linker.

Entre os dois: **geração de código intermédio**, que não depende nem da linguagem fonte nem da máquina destino.

Esta separação torna o compilador mais simples e permite **reutilizar componentes**: o *backend* do GCC serve para compilar C, C++ ou Objective-C; o *backend* LLVM é partilhado por Clang, Swift, Rust e (opcionalmente) GHC. Um mesmo *frontend* também pode, em teoria, ser usado com vários *backends* diferentes.

Este semestre o foco está sobretudo no **frontend** — é aí que entram a análise lexical (Aulas 2–3, abaixo) e a análise sintática.

Self-hosting e bootstrapping**Definição — Self-hosting**

Um compilador é **self-hosted** quando está escrito na própria linguagem que compila. Exemplos: GCC é escrito em C, Javac em Java, GHC em Haskell, Rustc em Rust.

Self-hosting é normalmente visto como prova de maturidade da linguagem e das suas ferramentas (“*eat your own dog food*”) e facilita a comunicação entre programadores do próprio compilador (só precisam de saber uma linguagem). O problema óbvio é: **se o compilador de X está escrito em X, o que compilou a primeira versão?**

Atenção — Bootstrapping

Resolve-se por **bootstrapping**: escreve-se uma primeira versão do compilador numa *outra* linguagem, usa-se essa versão para compilar uma segunda versão já escrita na linguagem-alvo, e repete-se o processo até o compilador conseguir compilar-se a si próprio.

- **C**: o primeiro compilador de C (1973) foi baseado no compilador de B (uma linguagem anterior); o primeiro compilador de B tinha sido escrito em TMG (uma linguagem do sistema Multics). O compilador foi depois reescrito em B, por estágios, até ser capaz de compilar a si próprio.
- **Rust**: o primeiro compilador (2006–2010) foi escrito em OCaml; a partir de 2010 começou a reescrita em Rust (ainda compilada pelo compilador OCaml); desde 2011 o compilador de Rust é *self-hosted*.

Aula 2 — Análise lexical

O que faz a análise lexical

O **léxico** de uma linguagem de programação é o conjunto de símbolos e palavras usados para compor programas — os *tokens*. Há várias categorias típicas: **identificadores** (nomes de variáveis/funções), **literais** (números, caracteres, cadeias), **palavras reservadas** (`if`, `else`, `while`, ...) e **operadores** (`+`, `*`, `=`, ...).

Um **analisador lexical** (também chamado *lexer*, *scanner* ou *tokenizer*) decompõe o texto de entrada numa sequência de *tokens*. Esta decomposição facilita a análise sintática seguinte — o *parser* trabalha sobre uma lista de *tokens* já classificados, não sobre texto em bruto carácter a carácter.

Definição — Representação de um token

Cada *token* é identificado por uma **etiqueta** (*token type*), e alguns tipos têm também um **valor** associado:

Entrada	Etiqueta	Valor
<code>if</code>	IF	—
<code>,</code>	COMMA	—
<code>!=</code>	NOTEQ	—
<code>main</code>	ID	"main"
<code>71</code>	NUM	71
<code>.5</code>	REAL	0.5

Exemplo resolvido

Para o código C `float match0(char *s) { if (!strcmp(s, "0.0", 3)) return 0.0; },` o analisador lexical produz a sequência:

FLOAT, ID("match0"), LPAREN, CHAR, STAR, ID("s"), RPAREN, LBRACE, IF, LPAREN, BANG, ID("strcmp"), LPAREN, ID("s"), COMMA, STRING("0.0"), COMMA, NUM(3), RPAREN, RPAREN, RETURN, REAL(0.0), SEMI, RBRACE, EOF

Repara que **espaços, mudanças de linha, tabulações e comentários não geram tokens** — são consumidos e descartados pelo analisador (servem só para separar tokens). Um token EOF marca explicitamente o fim da entrada.

Expressões regulares

Escrever um analisador lexical à mão, carácter a carácter (com um `switch` gigante, por exemplo), é simples mas repetitivo, difícil de manter e fácil de errar (ex: é preciso ter cuidado com a ordem das condições — "`if`" tem de ser reconhecido como palavra reservada e não como identificador). A alternativa é **descrever os tokens de forma declarativa** usando expressões regulares, e depois gerar automaticamente o analisador a partir dessa descrição — é isso que os geradores de analisadores (Aula 3) fazem.

Definição — Expressões regulares (REs)

Dado um alfabeto Σ , uma **linguagem** é um subconjunto $L \subseteq \Sigma^*$ de palavras formadas a partir de Σ . As expressões regulares constroem-se assim:

- a — uma letra $a \in \Sigma$ (reconhece só a palavra " a ")
- ε — a palavra vazia
- $M \mid N$ — **união**: reconhece o que M ou N reconhecem
- $M \cdot N$ — **concatenação**: palavras $u \cdot v$ tais que $u \in L(M)$ e $v \in L(N)$
- M^* — **repetição** (fecho de Kleene): concatenação de zero ou mais palavras de M

Definição — Convenções e abreviaturas

O símbolo de concatenação escreve-se muitas vezes de forma implícita: $abc = (a \cdot b) \cdot c$ (não é ambíguo, por associatividade). A prioridade dos operadores é **repetição** > **concatenação** > **união**, ou seja $ab|c = (ab)|c$ e $abb^* = ab(b^*)$.

Abreviaturas comuns:

Notação	Significado
$M?$	opcional: $M \mid \varepsilon$
M^+	uma ou mais vezes: $M \cdot M^*$
$[x_1x_2 \dots x_n]$	alternativa entre caracteres: $x_1 \ x_2 \ \dots \ x_n$
$[a-z]$	intervalo de caracteres contíguos
$[^a-z]$	complementar: qualquer carater fora do intervalo
$\cdot$	qualquer carater exceto mudança de linha
"a+"	cadeia literal (entre aspas, sem interpretar + como operador)

Exemplo resolvido

Descrever os *tokens* de um pequeno analisador em C:

```
if                palavra reservada (IF)
[_a-zA-Z][_a-zA-Z0-9]*  identificadores (ID)
[0-9]+            número inteiro (NUM)
([0-9]+|" [0-9]*")|([0-9]*" [0-9]+)  número fracionário (REAL)
//.*             comentário até ao final da linha
```

Note-se que `[_a-zA-Z][_a-zA-Z0-9]*` também reconheceria `if` — por isso a ordem/prioridade importa: as regras dos geradores (Aula 3) tratam este caso com a regra do “*first match*”, colocando `if` antes do padrão de identificador.

Atenção

Nota adicional (não estava explícito nos slides, mas é a razão pela qual o Exercício 2(c) do laboratório avisa que os comentários multi-linha “são mais difíceis do que parece”): à primeira vista pareceria bastar `/*.*.*/` para reconhecer um comentário `/* ... */`. O problema é que `.` costuma **incluir tudo menos a mudança de linha**, e `.*` é *greedy* — tentará consumir o máximo de caracteres possível. Numa entrada como `/* a */ x /* b */`, essa expressão reconheceria a linha inteira como **um único** comentário (desde o primeiro `/*` até ao **último** `*/`), em vez de dois comentários separados. A técnica geral para “parar no primeiro terminador” é excluir explicitamente o carácter do terminador do meio da expressão, por exemplo (usando `<!--/-->` como ilustração diferente do exercício, para não to resolver): um comentário `<!-- conteúdo -->` descreve-se como `"<!--" . ([^-]|"-"[^-])* . "-->"` — “qualquer carácter que não seja `-`, ou um `-` que não seja seguido de outro `-`”, até fechar com `-->`. A mesma ideia aplica-se ao `/* ... */` do exercício, adaptada ao terminador `*/`.

Pratica agora — Expressões regulares (Folha lab. 2, Exercício 2)

Papel e lápis, usando só esta secção:

- (a) — expressões regulares para todos os tokens do `scanner-c` (ID, NUM, LPAREN, RPAREN, COMMA, IF e os do Exercício 1: chavetas, `;`, WHILE, FOR, INT, FLOAT, REAL). Não te esqueças do `_` nos identificadores.
- (b) — generaliza REAL para notação científica (12.34e+12, 1e-12, 123.4E+9): repara que em 1e-12 não há ponto.
- (c) — comentários `// ...` e `/* ... */`. Para o multi-linha usa a técnica da caixa de atenção acima (a do `<!-- -->`), adaptada ao terminador `*/`.

Soluções, uma alínea de cada vez: [SOLUCOES_Compiladores.html](#)

Autómatos finitos determinísticos (DFA)

As expressões regulares são **descrições declarativas**: dizem o que reconhecer, não como. Para implementar um analisador precisamos de um **modelo operacional** — um autômato finito.

Definição — DFA

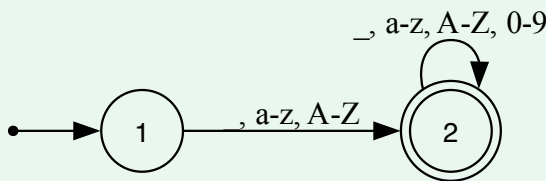
Um **autômato finito determinístico** é um quintuplo $(\Sigma, Q, q_0, F, \delta)$:

- Σ — alfabeto
- Q — conjunto finito de estados
- $q_0 \in Q$ — estado inicial (único)
- $F \subseteq Q$ — conjunto de estados finais (de aceitação)
- $\delta \subseteq Q \times \Sigma \times Q$ — transições, **determinísticas**: para cada $q \in Q$ e $\sigma \in \Sigma$ existe **no máximo um** q' tal que $(q, \sigma, q') \in \delta$. Também se pode ver δ como uma função parcial $q' = \delta(q, \sigma)$.

Uma palavra é **aceite** se, partindo de q_0 e seguindo as transições para cada símbolo da palavra, se terminar num estado de F .

Exemplo resolvido

DFA equivalente à expressão regular de identificadores `[_a-zA-Z][_a-zA-Z0-9]*`:



Formalmente: Σ = caracteres ASCII, $Q = \{1, 2\}$, $q_0 = 1$, $F = \{2\}$, e δ contém $(1, \ell, 2)$ para toda a letra/underscore ℓ , e $(2, \ell, 2)$, $(2, d, 2)$ para toda a letra/underscore ℓ e dígito d .

Percorrendo `x1`: $1 \xrightarrow{x} 2 \xrightarrow{1} 2$. Como $2 \in F$, a palavra é **aceite**.

Autómatos finitos não-determinísticos (NFA)

Definição — NFA

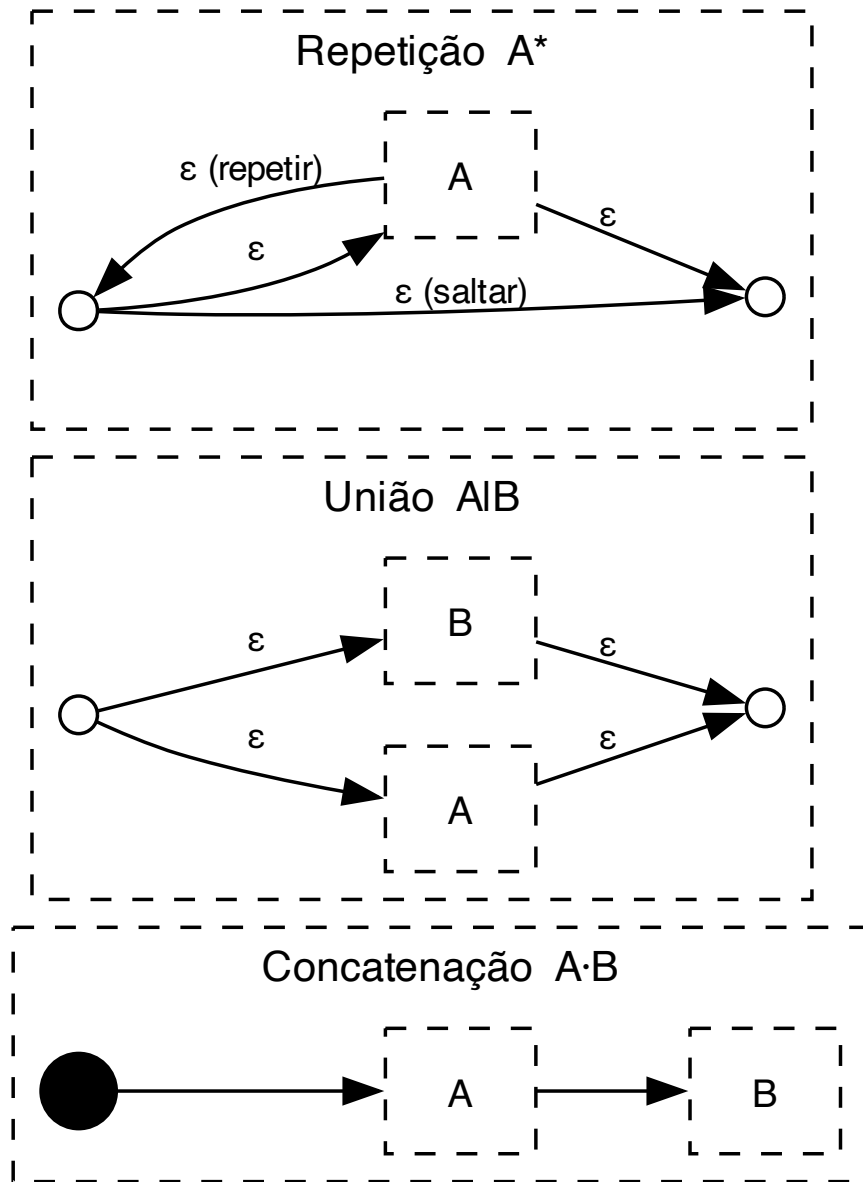
Um **NFA** tem a mesma estrutura que um DFA, mas as transições podem ser **não-determinísticas**: $\delta \subseteq Q \times (\Sigma \cup \{\varepsilon\}) \times Q$. Isto permite duas coisas que um DFA não tem:

- **várias transições** com o mesmo símbolo a partir do mesmo estado (ex: de 1, duas transições distintas com `a`, para 2 e para 3);
- **transições- ε** , que mudam de estado **sem consumir** nenhum símbolo de entrada.

Um NFA não se pode implementar diretamente da mesma forma que um DFA (nunca sabemos, num dado momento, “em que estado estamos” — pode haver vários possíveis ao mesmo tempo, e as transições- ε complicam ainda mais a leitura carácter a carácter). Por isso o percurso normal é: **expressão regular** $\rightarrow$ **NFA** (fácil) $\rightarrow$ **DFA equivalente** (mais complexo, mas implementável diretamente).

Construção de Thompson: de expressão regular a NFA

A construção de Thompson traduz uma expressão regular num NFA de forma composicional: cada sub-expressão vira um **fragmento** de autômato com uma única entrada e uma única saída, e os fragmentos combinam-se seguindo a estrutura da expressão.



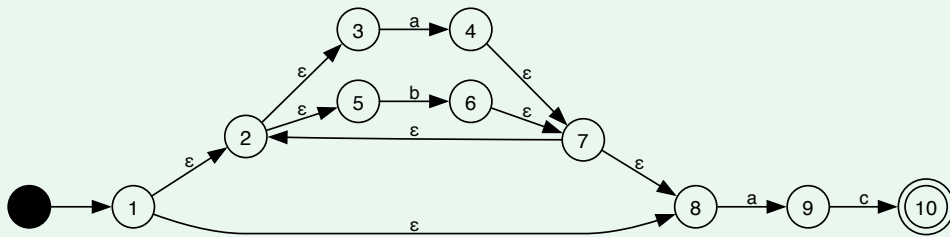
Definição — Regras de Thompson

- **Símbolo** $\sigma \in \Sigma$: um fragmento com uma única transição rotulada σ da entrada para a saída (e igual para ε).
- **Concatenação** $A \cdot B$: a saída do fragmento de A liga-se diretamente à entrada do fragmento de B .
- **União** $A \mid B$: um novo estado de entrada liga-se, por ε , às entradas de A e de B ; as saídas de A e de B ligam-se, por ε , a um novo estado de saída comum.
- **Repetição** A^* : um novo par entrada/saída; a entrada liga-se por ε à entrada de A (para iterar) e diretamente à saída (para “saltar” zero repetições); a saída de A liga-se por ε de volta à sua própria entrada (para repetir) e à nova saída (para terminar).

Exemplo resolvido

Construir o NFA para $(a \mid b)^* \cdot a \cdot c$ (o exemplo da aula), passo a passo:

1. Fragmento para **a** (estados 3→4) e fragmento para **b** (estados 5→6).
2. **União** $a \mid b$: novo estado 2 (entrada, ε para 3 e para 5) e novo estado 7 (saída, ε a partir de 4 e de 6).
3. **Repetição** $(a \mid b)^*$: novo par 1 (entrada) / 8 (saída); $1 \rightarrow 2$ (entrar no corpo), $1 \rightarrow 8$ (saltar, zero repetições), $7 \rightarrow 2$ (repetir), $7 \rightarrow 8$ (terminar o ciclo).
4. **Concatenação** com **a**: a saída do $*$ (estado 8) liga-se por **a** a um novo estado 9.
5. **Concatenação** com **c**: o estado 9 liga-se por **c** ao estado final 10.

**Conversão de NFA em DFA: construção de subconjuntos****Definição — closure e construção de subconjuntos**

Seja $\mathcal{A} = (\Sigma, Q, q_0, F, \delta)$ um NFA. Define-se

$$\text{closure}(S) = \{ q' \in Q : \text{existe } q \in S \text{ e um caminho de transições-}\varepsilon \text{ entre } q \text{ e } q' \}$$

(ou seja, todos os estados alcançáveis a partir de S **sem consumir nenhum símbolo**). O NFA transforma-se no DFA equivalente $\mathcal{A}' = (\Sigma, 2^Q, S_0, F', \delta')$, onde **cada estado do DFA é um conjunto de estados do NFA**:

$$S_0 = \text{closure}(\{q_0\}) \quad F' = \{ S \subseteq Q : S \cap F \neq \emptyset \} \quad \delta'(S, \sigma) = \text{closure}(\{ q' : q \in S \wedge (q, \sigma, q') \in \delta \})$$

Como $\mathcal{A}'$ é determinístico, δ' pode ver-se como função.

Exemplo resolvido

Aplicando a construção de subconjuntos ao NFA de $(a \mid b)^*ac$ construído acima. Transições- ε do NFA (para calcular *closures*): $1 \rightarrow 2$, $1 \rightarrow 8$, $2 \rightarrow 3$, $2 \rightarrow 5$, $4 \rightarrow 7$, $6 \rightarrow 7$, $7 \rightarrow 2$, $7 \rightarrow 8$. Transições com símbolo: $3 \xrightarrow{a} 4$, $5 \xrightarrow{b} 6$, $8 \xrightarrow{a} 9$, $9 \xrightarrow{c} 10$.

Estado inicial: $S_0 = \text{closure}(\{1\}) = \{1, 2, 3, 5, 8\}$ — partindo de 1, seguimos só ε : $1 \rightarrow 2$, $2 \rightarrow 3$, $2 \rightarrow 5$, $1 \rightarrow 8$; nenhum destes $(2, 3, 5, 8)$ tem mais ε -saídas, por isso o *closure* para aqui.

Calculamos agora, **estado a estado**, para onde vai S_0 com cada símbolo:

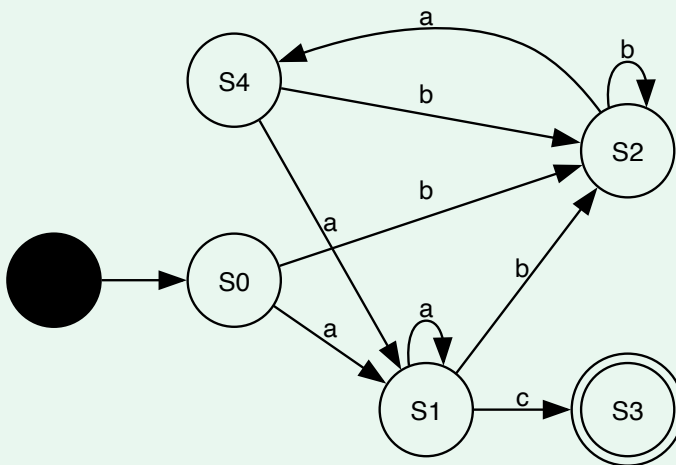
- $S_0 = \{1, 2, 3, 5, 8\}$ **com a**: só os estados 3 e 8 (de S_0) têm transição por **a** ($3 \xrightarrow{a} 4$ e $8 \xrightarrow{a} 9$), logo $\text{move} = \{4, 9\}$. $\text{closure}(\{4, 9\})$: de 4, ε para 7, depois $7 \rightarrow 2$ e $7 \rightarrow 8$, depois $2 \rightarrow 3$ e $2 \rightarrow 5$; de 9 não há mais ε . Total: $\{2, 3, 4, 5, 7, 8, 9\}$ — como este conjunto ainda não tinha aparecido, chamamos-lhe S_1 .
- S_0 **com b**: só o estado 5 tem transição por **b** ($5 \xrightarrow{b} 6$), logo $\text{move} = \{6\}$. $\text{closure}(\{6\})$: $6 \rightarrow 7$, depois $7 \rightarrow 2$, $7 \rightarrow 8$, depois $2 \rightarrow 3$, $2 \rightarrow 5$. Total: $\{2, 3, 5, 6, 7, 8\}$ — novo, chamamos-lhe S_2 .
- S_0 **com c**: nenhum estado de $S_0 = \{1, 2, 3, 5, 8\}$ tem transição por **c** (só o estado 9 tem, e $9 \notin S_0$) — **sem transição definida**.

Exemplo resolvido

Continuando a partir de $S_1 = \{2, 3, 4, 5, 7, 8, 9\}$ e $S_2 = \{2, 3, 5, 6, 7, 8\}$:

- S_1 **com a**: estados de S_1 com transição a: 3 e 8 (os mesmos de antes, ambos $\in S_1$) $\Rightarrow$ move = $\{4, 9\}$, closure = $\{2, 3, 4, 5, 7, 8, 9\} = S_1$ — **fica no mesmo estado** (*self-loop* em a).
- S_1 **com b**: estado 5 $\in S_1$ tem transição b $\Rightarrow$ move = $\{6\}$, closure = $\{2, 3, 5, 6, 7, 8\} = S_2$.
- S_1 **com c**: estado 9 $\in S_1$ tem transição c ($9 \xrightarrow{c} 10$) $\Rightarrow$ move = $\{10\}$, closure($\{10\}$) = $\{10\}$ (o 10 não tem ε -saídas) — novo, chamamos-lhe S_3 . Como 10 é o estado final do NFA, $S_3 \in F'$.
- S_2 **com a**: só o estado 3 $\in S_2$ tem transição a (o 8 **não** está em S_2 , por isso desta vez só contamos o 3) $\Rightarrow$ move = $\{4\}$, closure($\{4\}$): $4 \rightarrow 7, 7 \rightarrow 2, 7 \rightarrow 8, 2 \rightarrow 3, 2 \rightarrow 5$. Total: $\{2, 3, 4, 5, 7, 8\}$ — novo, chamamos-lhe S_4 .
- S_2 **com b**: estado 5 $\in S_2 \Rightarrow$ move = $\{6\}$, closure = $\{2, 3, 5, 6, 7, 8\} = S_2$ — *self-loop* em b.
- S_2 **com c**: nenhum estado de S_2 tem transição c — sem transição.
- $S_4 = \{2, 3, 4, 5, 7, 8\}$ **com a**: estados 3 e 8 $\in S_4 \Rightarrow$ move = $\{4, 9\}$, closure = $\{2, 3, 4, 5, 7, 8, 9\} = S_1$ (já conhecido).
- S_4 **com b**: estado 5 $\in S_4 \Rightarrow$ move = $\{6\}$, closure = $\{2, 3, 5, 6, 7, 8\} = S_2$ (já conhecido).
- S_4 **com c**: nenhum estado de S_4 tem transição c — sem transição.

Todas as transições de S_4 vão para estados já conhecidos (S_1, S_2) — **não aparece nenhum estado novo**, o processo termina. Resultado final: 5 estados S_0 a S_4 , com $F' = \{S_3\}$ (único conjunto que contém o estado final 10 do NFA):



Minimização de autómatos

O DFA obtido pela construção de subconjuntos pode ter **estados redundantes** — estados diferentes (como conjuntos) mas com exatamente o mesmo comportamento (as mesmas transições de saída para todo o alfabeto, e o mesmo estatuto de aceitação). Podemos “fundir” esses estados equivalentes e obter um **autômato mínimo** equivalente. Os geradores de analisadores lexicais (Alex, Flex — Aula 3) fazem esta minimização automaticamente.

Frequente em exame

Sai frequentemente em exame: reconhecer estados equivalentes. No exemplo acima, repara que $S_0 = \{1, 2, 3, 5, 8\}$ e $S_4 = \{2, 3, 4, 5, 7, 8\}$ têm exatamente as mesmas transições ($\xrightarrow{a} S_1$, $\xrightarrow{b} S_2$, sem transição em c) — a diferença entre os conjuntos (1 vs. $\{4, 7\}$) não afeta o comportamento, porque nenhum desses estados extra tem transições que consomem símbolos. Logo $S_0 \equiv S_4$: seriam fundidos na minimização, reduzindo o autômato de 5 para 4 estados. Este é exatamente o tipo de equivalência ($S_0 \equiv S_2$ no exemplo da aula) que costuma aparecer nos exercícios de exame sobre minimização.

Aula 3 — Geradores de analisadores lexicais

Recapitulando o percurso: **expressão regular** → **NFA (Thompson)** → **DFA (subconjuntos)** → **implementação**. Esta aula cobre o último passo: como implementar um analisador a partir de um DFA, e como **gerar** esse analisador automaticamente a partir das expressões regulares (sem passar manualmente pelos passos intermédios).

Implementação direta de um DFA em C

Um DFA pequeno pode implementar-se com uma **tabela de transições** (`delta[estado][símbolo]`) e um ciclo que lê caracteres e segue as transições:

Exemplo resolvido

Para o DFA de identificadores (secção anterior), com estado 0 reservado para representar “erro” (ausência de transição válida) e estado final 2:

```
#define STATES 3 // estados 0 (erro), 1, 2
#define SYMBOLS 128 // códigos ASCII
const int delta[STATES][SYMBOLS] = {
    /* estado 0 */ { 0, ..., 0, ... }, // erro é absorvente
    /* estado 1 */ { 0, ..., 2, ..., 2, ... 0 }, // letra/_ -> 2
    /* estado 2 */ { 0, ..., 2, ..., 2, ... 0 } }; // letra/_/dígito -> 2

int state = 1; // estado inicial
int c;
while ((c = getchar()) != EOF) {
    state = delta[state][c];
    if (state == 0) return ERROR; // rejeita
    if (FINAL(state)) return ACCEPT; // aceita (aqui, FINAL(state) é state==2)
}
return ERROR; // fim do input sem atingir estado final
```

Traçando a execução com a entrada “x1”:

Passo	c lido	delta[state][c]	novos state ação
1	x (letra)	delta[1]['x'] = 2	2 state != 0 (não rejeita); FINAL(2) é verdade → return ACCEPT imediatamente

O ciclo **nunca chega a ler o 1!** Isto revela uma limitação importante deste pseudo-código, que vale a pena perceber bem: ele aceita assim que atinge **qualquer** estado final, não espera para ver se consumindo mais caracteres continuaria num estado válido (não implementa *longest match*). Para este DFA de identificadores em particular isto até não causa erro de classificação (qualquer prolongamento de um identificador continua a ser um identificador válido), mas é exatamente o motivo por que os analisadores lexicais reais (Aula 3, secção “Longest match e first match”, abaixo) têm de **guardar a posição do último estado final visto e continuar a tentar consumir mais caracteres**, só decidindo aceitar quando deixar de haver transição possível — este pseudo-código simplificado dos slides ilustra a ideia da tabela de transições, mas não implementa essa regra.

Isto funciona como ilustração da tabela de transições, mas é **específico de um único DFA** — cada novo token exigiria escrever/combinar mais tabelas à mão. É exatamente este trabalho mecânico que os geradores (abaixo) automatizam a partir de expressões regulares.

Prática agora — DFA em C (Folha lab. 2, Exercício 1)

O `src/CLexer.c` do `scanner-c` segue o padrão “ler carácter, decidir por casos” desta secção (um `switch/while`, sem tabela explícita). Estende-o:

- (a), (b) — novos tokens `{, }, ;` e palavras reservadas `WHILE, FOR, INT, FLOAT`: o mesmo padrão de `IF/LPAREN` já lá.
- (c) — aceitar `_` como letra: é a expressão regular `[_a-zA-Z]` mal transcrita para o C (falta um `case '_'`: a par de `isalpha(c)`).
- (d) — `REAL (0.5, 123.45)`: um estado novo depois do `..`
- (e) — ignorar comentários `/* ... */` e `// ....`
- (f) — *buffer overrun* nos identificadores: testa o comprimento máximo antes de escrever no buffer.

Cuidado com a ordem `if` vs. `ID` (ver “Longest match e first match”).

Soluções, uma alínea de cada vez: [SOLUCOES_Compiladores.html](#)

Alex (gerador para Haskell)

Definição — Alex

O **Alex** é um gerador de analisadores lexicais para Haskell. Recebe um ficheiro de descrição (ex: `Lexer.x`) com regras “expressão regular → ação em Haskell” e definições auxiliares, e produz um ficheiro `Lexer.hs` que pode ser compilado com o resto do projeto.

`Lexer.x` $\xrightarrow{\text{alex}}$ `Lexer.hs` $\xrightarrow{\text{compilador Haskell}}$ executável

Exemplo resolvido

```
{
module Lexer where
}
%wrapper "basic"
$alpha = [_a-zA-Z]
$digit = [0-9]

tokens :-
$white+           ; -- ignorar caracteres brancos
if                { \_ -> IF }
$alpha($alpha|$digit)*  { \s -> ID s }
$digit+           { \s -> NUM (read s) }
$digit+"."$digit+    { \s -> REAL (read s) }
```

```
{
data Token = IF | ID String | NUM Int | REAL Double
}
```

As partes entre `{ }` são código Haskell; o resto são diretivas, definições e padrões. Cada regra tem a forma `padrão { ação }`, onde o padrão é uma expressão regular e a ação é uma função `String -> Token`. Usar: `Lexer.hs` exporta `alexScanTokens :: String -> [Token]`.

```
module Main where
import Lexer
main = do
  txt <- getContents
  print (alexScanTokens txt)
```

O exemplo acima usa a interface `%wrapper "basic"`. Existe também `%wrapper "posn"`, que associa a cada

token a sua **posição** (linha e coluna) — útil para reportar erros com precisão. Nesse caso, cada ação passa a ter tipo `AlexPosn -> String -> Token`, com data `AlexPosn = AlexPn Int Int Int` (posição absoluta, linha, coluna), e o padrão de cada regra recebe também `pos` (ex: `\pos s -> ID pos s`).

Flex (gerador para C)

Definição — Flex

O **Flex** é o equivalente do Alex mas para C. Recebe um ficheiro `.l` (ex: `lexer.l`) e produz `lex.yy.c`, compilável com o resto do projeto C.

$$\text{ficheiro.l} \xrightarrow{\text{flex}} \text{lex.yy.c} \xrightarrow{\text{compilador C}} \text{executável}$$

Exemplo resolvido

```
%{
#include "tokens.h"
%}
%option noyywrap
alpha      [_a-zA-Z]
digit      [0-9]
%%

[ \t\n\r]+      /* skip whitespace */
if               { return IF; }
{alpha}{alpha}|{digit}* { return ID; }
{digit}+        { yynum = atoi(yytext); return NUM; }
{digit}+"."{digit}+ { yyreal = atof(yytext); return REAL; }
<<EOF>>         { return EOF; /* end of input */ }
```

Estrutura geral de um ficheiro Flex: `%{` declarações `%}`, depois definições auxiliares, `%%`, depois as regras padrão `{ ação em C }`, e opcionalmente mais `%%` seguido de código C auxiliar.

```
#include "tokens.h"
int main(void) {
    int token;
    while ((token = yylex()) != EOF) {
        switch (token) {
            case IF:   printf("IF ");           break;
            case ID:   printf("ID(%s) ", yytext); break;
            case NUM:  printf("NUM(%d) ", yynum); break;
            case REAL: printf("REAL(%f) ", yyreal); break;
        }
    }
}
```

A função `yylex()` devolve o próximo token (um inteiro, definido num ficheiro *header*). A variável global `yytext` contém sempre o texto do token atual — é assim que se acede ao valor (ex: o nome de um identificador). Os valores de tokens não-textuais (como `NUM`, `REAL`) costumam guardar-se em variáveis de estado globais (`yynum`, `yyreal`).

Atenção

Nota adicional: por omissão o Flex **não** guarda a posição de cada token. Para isso usa-se `%option yylineno`, que mantém a variável global `yylineno` com o número da linha atual — mas é **responsabilidade do programador** ir guardando essa posição à medida que os tokens são consumidos (ao contrário do Alex com `%wrapper "posn"`, que já entrega a posição em cada ação).

Longest match e first match**Definição — Regras de desambiguação**

Tanto o Alex como o Flex (e geradores semelhantes) usam duas regras, por esta ordem, para decidir qual regra aplicar quando várias expressões regulares poderiam casar com o *input*:

1. **Longest match**: escolhe-se a regra que consome o **maior número de caracteres** de entrada.
2. **First match**: em caso de empate no comprimento, escolhe-se a regra que aparece **primeiro** no ficheiro de descrição.

Frequente em exame

Consequência prática (e fonte comum de bugs/perguntas de exame): identificadores e palavras reservadas partilham o mesmo texto possível (`if` casa tanto com o padrão literal `if` como com o padrão de identificador `[_a-zA-Z][_a-zA-Z0-9]*`, com o **mesmo comprimento** — 2 caracteres). Por *first match*, a regra que resolve a ambiguidade corretamente é **colocar os padrões mais específicos (palavras reservadas) antes dos mais gerais (identificadores)** no ficheiro `.x/ .l`. Se a ordem for trocada, `if` seria sempre lido como ID, nunca como IF.

Prática agora — Flex (Folha lab. 2, Exercício 3)

- Reimplementa o `scanner-c` num ficheiro `Lexer.l`, com a estrutura de três partes da secção “Flex” acima e as expressões regulares que escreveste no Exercício 2. Compila com `flex Lexer.l` e `gcc lex.yy.c -o Lexer`.
- Põe as palavras reservadas **antes** da regra de ID (*first match*, caixa acima) e confirma que `iffy` sai como ID (*longest match*).

Soluções, uma alínea de cada vez: SOLUCOES_Compiladores.html

Comparação rápida Alex vs. Flex

	Alex	Flex
Linguagem gerada	Haskell (<code>.hs</code>)	C (<code>lex.yy.c</code>)
Ficheiro de entrada	<code>.x</code>	<code>.l</code>
Acesso ao texto do token	argumento da ação (<code>\s -> ...</code>)	variável global <code>yytext</code>
Posição do token	<code>%wrapper "posn"</code> (automático, por ação)	<code>%option yylineno</code> (manual)
Função de arranque	<code>alexScanTokens :: String -> [Token]</code>	<code>yylex()</code> (chamado em ciclo)

Aula 4 — Análise sintática e gramáticas independentes de contexto

As Aulas 2–3 cobriram a **análise lexical**: transformar texto em bruto numa sequência de *tokens*. Esta aula avança para o passo seguinte do *frontend* (ver o diagrama de fases na Aula 1, secção “Fases de um compilador” — a análise sintática é a fase logo a seguir à lexical): a **análise sintática**, que verifica se a sequência de tokens respeita a estrutura gramatical da linguagem e constrói uma **árvore sintática**.

O que é a análise sintática

Definição — Sintaxe

Sintaxe: “estudo das regras e dos princípios que regem a organização dos constituintes das frases” (Dicionário Priberam). Aplicado a uma linguagem de programação: um programa é **sintaticamente bem formado** se respeitar regras de estrutura — chavetas { } e parêntesis () “casados”, operadores com o número correto de operandos, instruções terminadas ou separadas corretamente (;), etc.

Tal como na linguagem natural, um programa pode estar sintaticamente correto e ainda assim não fazer sentido — o exemplo clássico de Chomsky (1957) é a frase em inglês “*Colorless green ideas sleep furiously*” (gramaticalmente válida, semanticamente absurda). Verificar o “faz sentido” é trabalho da **análise semântica** (fase seguinte, fora do âmbito desta aula); a análise sintática só verifica estrutura.

O **analisador sintático** (*parser*) constrói a árvore sintática a partir da sequência de tokens (ou reporta um erro, se a sequência não respeitar nenhuma estrutura válida). A ferramenta matemática usada para especificar essa estrutura é a **gramática independente de contexto**.

Gramáticas independentes de contexto

Definição — Gramática independente de contexto (CFG)

Uma **gramática independente de contexto** é um quádruplo $G = (\Sigma, N, S, P)$:

- Σ — conjunto de símbolos **terminais** (os tokens — os mesmos que saem do analisador lexical);
- N — conjunto de símbolos **não-terminais** (categorias sintáticas abstratas, ex: “expressão”, “instrução”);
- $S \in N$ — símbolo **inicial**;
- P — conjunto de **produções** da forma $X \rightarrow \alpha$, onde X é um não-terminal e α é uma sequência (possivelmente vazia) de terminais e/ou não-terminais.

Chama-se “independente de contexto” porque o lado esquerdo de cada produção é **sempre um único não-terminal isolado** (nunca depende do que está à sua volta na frase para saber se a produção se pode aplicar — ao contrário de gramáticas mais gerais, fora do âmbito deste curso).

Exemplo resolvido

Gramática usada como fio condutor ao longo desta secção (é a gramática usada nos slides para introduzir derivações, árvores e ambiguidade — vai reaparecer várias vezes abaixo, sempre com a mesma numeração de produções):

$$\Sigma = \{a, b\} \quad N = \{S, B\} \quad \text{inicial: } S$$

Produções:

Nº	Produção
(1)	$S \rightarrow aSB$
(2)	$S \rightarrow \varepsilon$
(3)	$S \rightarrow B$
(4)	$B \rightarrow Bb$
(5)	$B \rightarrow b$

Derivações**Definição — Derivação**

A relação de **derivação** $\Rightarrow$ substitui um não-terminal pelo lado direito de alguma produção que o tenha como lado esquerdo. $\Rightarrow^*$ (fecho transitivo) representa zero ou mais passos de derivação.

Exemplo resolvido

Derivação completa de $aabbb$ a partir de S , usando a gramática acima, anotando cada passo com o número da produção aplicada:

$$S \xRightarrow{1} aSB \xRightarrow{1} aaSBB \xRightarrow{2} aaBB \xRightarrow{4} aaBbB \xRightarrow{5} aabbB \xRightarrow{5} aabbb$$

Passo a passo: (1) $S \rightarrow aSB$ substitui o S inicial $\rightarrow aSB$. (1) novamente, no S que sobrou $\rightarrow aaSBB$ (repara que a produção acrescenta sempre um a à esquerda e um B à direita). (2) $S \rightarrow \varepsilon$ faz o S desaparecer $\rightarrow aaBB$ (dois a 's, dois B 's por resolver). (4) $B \rightarrow Bb$ no primeiro $B \rightarrow aaBbB$ (esse B vira Bb , ou seja um B novo seguido de b). (5) $B \rightarrow b$ no B que sobrou do passo anterior $\rightarrow aabbB$. (5) outra vez, no último $B \rightarrow aabbb$. Todos os não-terminais foram eliminados: $aabbb$ é uma palavra da linguagem.

Atenção — Derivação mais à esquerda e mais à direita

Nota adicional (não estava explícito nos slides, mas a Aula 5 usa o nome “*Rightmost derivation*” sem o definir): quando uma forma intermédia tem vários não-terminais, podemos escolher qual substituir primeiro.

- **Derivação mais à esquerda** (*leftmost*): em cada passo substitui-se sempre o não-terminal **mais à esquerda**. A derivação de `aabbb` acima é deste tipo (em `aaBB` expandiu-se primeiro o `B` da esquerda).
- **Derivação mais à direita** (*rightmost*): em cada passo substitui-se sempre o não-terminal **mais à direita**. Para a mesma árvore de `aabbb`:

$$S \xRightarrow{1} aS\underline{B} \xRightarrow{5} a\underline{S}b \xRightarrow{1} aaS\underline{B}b \xRightarrow{4} aaS\underline{B}bb \xRightarrow{5} aa\underline{S}bbb \xRightarrow{2} aabbb$$

(sublinhado: o não-terminal mais à direita, que é o próximo a ser substituído). Os dois `B` são resolvidos antes do `S` do meio, porque estão à direita dele.

As duas derivações dão a **mesma árvore**; só muda a ordem dos passos. A análise *top-down* (LL) constrói uma derivação mais à esquerda; a análise *bottom-up* (LR, Aula 5) constrói uma derivação mais à direita, mas **de trás para a frente**.

Linguagem descrita por uma gramática**Definição — $L(G)$**

Partindo do símbolo inicial e substituindo não-terminais pelas produções até só restarem terminais, obtemos uma **palavra** descrita pela gramática. Formalmente, para $G = (\Sigma, N, S, P)$:

$$L(G) = \{ w \in \Sigma^* : S \Rightarrow^* w \}$$

Exemplo resolvido — Exercício 1(a) (Folha lab. 3, gramatica.pdf)

Exercício dos slides e da Folha 3, resolvido por completo: que linguagem descreve a gramática do exemplo acima ($S \rightarrow aSB \mid \varepsilon \mid B$; $B \rightarrow Bb \mid b$)? Onde podem ocorrer a's e b's numa palavra aceite, e qual a relação entre o número de a's e de b's?

Primeiro, o que faz cada não-terminal:

- B só gera seqüências de **um ou mais b's**: $B \rightarrow b$ dá **b**; $B \rightarrow Bb$ acrescenta sempre mais um **b** à direita de um B já formado. Logo B gera exatamente a linguagem b^+ (nunca b^0 , porque não há produção $B \rightarrow \varepsilon$).
- $S \rightarrow aSB$ aplicada n vezes seguidas produz $a^n S B_1 B_2 \cdots B_n$ (n cópias de B , cada uma independente das outras — ver a derivação acima, onde duas aplicações de (1) geram exatamente dois B 's pendentos). Cada B_i resolve-se depois, independentemente, num b^{k_i} com $k_i \geq 1$.
- O S que sobra no meio tem de terminar de duas formas possíveis:
 - via (2) $S \rightarrow \varepsilon$: não acrescenta mais nada. Com n cópias de B já geradas, o total de b's é $m = \sum_{i=1}^n k_i$, e como cada $k_i \geq 1$, temos $m \geq n$ (com $m = n$ exatamente quando todos os $k_i = 1$, e m pode crescer arbitrariamente aumentando qualquer k_i — logo **todos** os valores $m \geq n$ são atingíveis). Caso particular $n = 0$: dá a palavra vazia.
 - via (3) $S \rightarrow B$: acrescenta mais **um** B extra (o $(n+1)$ -ésimo), que por si só já gera b^k com $k \geq 1$. Isto dá $m \geq n + 1$ — mas este intervalo já está contido no anterior (para o mesmo n , a via (2) já atinge qualquer $m \geq n$, incluindo $m = n + 1, n + 2, \dots$), por isso não acrescenta palavras novas à linguagem, só dá origem a **derivações alternativas** para palavras que já eram atingíveis — é precisamente esta segunda via que torna a gramática **ambígua** (ver secção seguinte).

Juntando tudo: n pode ser qualquer natural ≥ 0 , e para cada n , m (número de b's) pode ser qualquer natural $\geq n$. Logo:

$$L(G) = \{ a^n b^m : n \geq 0, m \geq n \}$$

Em palavras (a resposta pedida “numa frase” pelo exercício): **a linguagem das palavras formadas por zero ou mais a's seguidos de zero ou mais b's, em que o número de b's nunca é inferior ao número de a's** (incluindo a palavra vazia, quando $n = m = 0$). Todos os a's aparecem sempre à esquerda de todos os b's — a gramática nunca permite intercalar os dois símbolos.

Escrever uma gramática para uma linguagem

O exercício inverso: dada uma linguagem, escrever uma gramática que a gere. Não há um algoritmo único, mas quando a linguagem vem dada por uma **expressão regular** há uma tradução mecânica.

Atenção — De expressão regular para gramática

Nota adicional (não estava explícito nos slides, mas a Folha 3 pede-o): cada construção de uma expressão regular (Aula 2) tem uma tradução direta em produções. Usa-se um não-terminal novo para cada sub-expressão com $*$, $+$ ou $|$.

Expressão regular	Produções
rs (concatenação)	um único lado direito com r seguido de s
$r s$ (alternativa)	duas produções: $X \rightarrow r$ e $X \rightarrow s$
r^* (zero ou mais)	$X \rightarrow rX \mid \varepsilon$
r^+ (uma ou mais)	$X \rightarrow rX \mid r$

Isto mostra que **toda a linguagem regular é independente de contexto**. O contrário não é verdade: $\{a^n b^m : m \geq n\}$ (a linguagem do exemplo acima) não é regular, porque um autômato finito não consegue contar os a 's para garantir que há pelo menos tantos b 's.

Exemplo resolvido — Exercício 1(d) (Folha lab. 3, gramatica.pdf)

Escrever uma gramática para a linguagem de $((ab^*a)|(ba^*b))$.

1. Separar pela alternativa de topo. A expressão é ab^*a ou ba^*b . Isso dá duas produções para o símbolo inicial: $S \rightarrow (\text{algo para } ab^*a)$ e $S \rightarrow (\text{algo para } ba^*b)$.

2. Tratar ab^*a . É uma concatenação de três partes: a , b^* e a . A parte b^* precisa de um não-terminal próprio, B , com a regra do $*$: $B \rightarrow bB \mid \varepsilon$. A concatenação fica $S \rightarrow aBa$.

3. Tratar ba^*b da mesma forma: $A \rightarrow aA \mid \varepsilon$ gera a^* , e $S \rightarrow bAb$.

Gramática final (terminais $\{a, b\}$, inicial S):

$$S \rightarrow aBa \mid bAb \quad B \rightarrow bB \mid \varepsilon \quad A \rightarrow aA \mid \varepsilon$$

Verificação com uma palavra: $abba$ deve ser aceite (é a , bb , a). Derivação: $S \Rightarrow aBa \Rightarrow abBa \Rightarrow abbBa \Rightarrow abba$ (as duas primeiras expansões de B usam $B \rightarrow bB$ e a última usa $B \rightarrow \varepsilon$). E ab não deve ser aceite: começa por a , por isso só pode vir de $S \rightarrow aBa$, que obriga a acabar em a .

Pratica agora — Escrever gramáticas (Folha lab. 3, Exercício 1)

Ex. 1(b) Gramática para as palavras com o mesmo número de a s e b s, por qualquer ordem. Aqui não há expressão regular (a linguagem não é regular). Pista: olha para a **primeira** letra da palavra e para o ponto onde as contagens voltam a ficar iguais pela primeira vez.

Ex. 1(c) Gramática para os parêntesis casados $()$, $(())$, $()()$, $\dots$). Pista: uma palavra não vazia começa por um $($ que fecha num certo $)$; o que está lá dentro e o que vem a seguir são outra vez parêntesis casados.

Ex. 1(e) Gramática para $((0|1)^+ \cdot "(0|1)^* | ((0|1)^* \cdot "(0|1)^+)$. Usa a tabela acima, como no exemplo da 1(d). Um não-terminal para “um dígito” $(0|1)$ simplifica tudo.

A 1(a) está resolvida acima e a 1(d) no exemplo anterior.

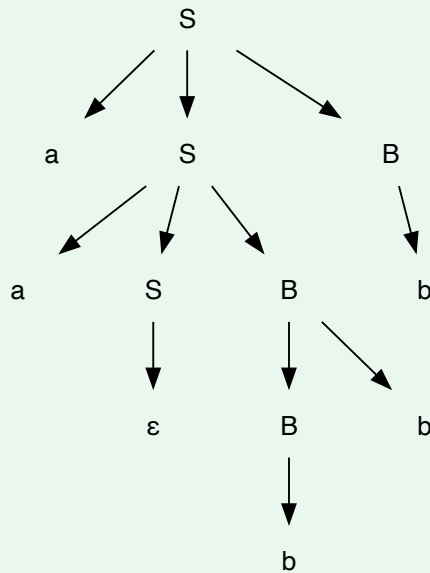
Soluções, uma alínea de cada vez: [SOLUCOES_Compiladores.html](#)

Árvores sintáticas**Definição — Árvore sintática**

Cada passo de uma derivação pode representar-se como um nó numa **árvore sintática**: uma produção $X \rightarrow \alpha_1 \dots \alpha_n$ corresponde a um nó X com n sub-árvores, uma por cada símbolo de $\alpha_1, \dots, \alpha_n$ (por ordem, da esquerda para a direita).

Exemplo resolvido

Árvore sintática correspondente à derivação de $aabbb$ feita acima ($S \xRightarrow{1} aSB \xRightarrow{1} aaSBB \xRightarrow{2} aaBB \xRightarrow{4} aaBbB \xRightarrow{5} aabbB \xRightarrow{5} aabbb$):



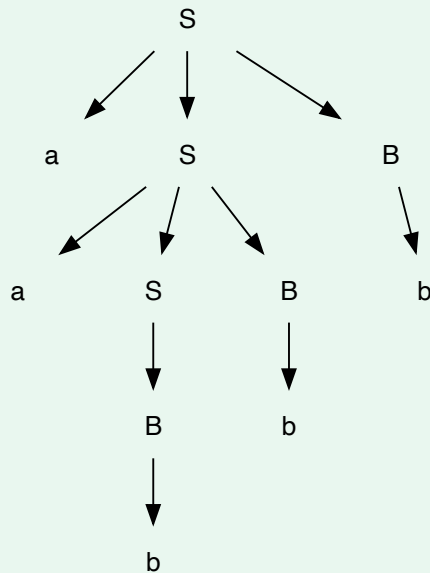
Lendo a árvore de cima para baixo: a raiz S usa a produção (1) e tem três filhos — a , o S do meio, e o B mais à direita (o “ B exterior”). O S do meio usa (1) outra vez — três filhos: a , o S mais interno, e um segundo B (o “ B do meio”). O S mais interno usa (2) e resolve-se em ε (fim da recursão). O “ B do meio” usa (4) $B \rightarrow Bb$ — dois filhos: um novo B , e b ; esse novo B usa (5) $B \rightarrow b$. O “ B exterior” usa diretamente (5) $B \rightarrow b$. Lendo as folhas da esquerda para a direita: a , a , (nada, do ε), b (do B novo dentro do B do meio), b (do próprio B do meio), b (do B exterior) — dá exatamente $aabbb$.

Gramáticas ambíguas**Definição — Ambiguidade**

Uma gramática diz-se **ambígua** se existe pelo menos uma palavra da sua linguagem que admite **duas (ou mais) árvores sintáticas distintas**. Nota: derivações diferentes podem corresponder à **mesma** árvore (diferindo só na ordem em que os não-terminais são substituídos) — o que importa para a ambiguidade é a árvore, não a derivação.

Exemplo resolvido

A mesma gramática do exemplo acima é **ambígua**: a palavra **aabbb** também se obtém pela derivação alternativa $S \xRightarrow{1} aSB \xRightarrow{1} aaSBB \xRightarrow{3} aaBBB \xRightarrow{5} aabBB \xRightarrow{5} aabbB \xRightarrow{5} aabbb$ — que usa (3) $S \rightarrow B$ no S mais interno (em vez de (2) $S \rightarrow \varepsilon$), dando **três** B 's independentes, cada um resolvido diretamente por (5) $B \rightarrow b$, sem nenhum usar (4) $B \rightarrow Bb$:



Esta árvore é **estruturalmente diferente** da anterior (a primeira tinha um B que se expandia em Bb ; esta tem três B 's todos “folha única” via (5)) apesar de produzir a mesma palavra **aabbb** — por definição, isto basta para a gramática ser ambígua. Esta é exatamente a razão, identificada na secção anterior, por que a via (3) $S \rightarrow B$ não alarga a linguagem: só dá um caminho alternativo para palavras já alcançáveis por (2).

Frequente em exame

Porque é que a ambiguidade importa? Como modelo matemático para *descrever* uma linguagem, uma gramática ambígua é perfeitamente válida. Mas num compilador a gramática também serve para **atribuir significado** aos fragmentos do programa (é a árvore sintática que a análise semântica e a geração de código percorrem depois) — se a mesma palavra tem duas árvores possíveis, o compilador não sabe qual significado escolher. Por isso as linguagens de programação são desenhadas para que a sua gramática não seja ambígua (ou, quando é impraticável evitar completamente, para que existam regras explícitas de desambiguação — ver “dangling else” abaixo). Este é um tema clássico de exame: identificar que uma gramática é ambígua exibindo **duas árvores diferentes** para a mesma palavra (não basta exibir duas derivações — têm de corresponder a árvores diferentes).

Exemplo central: expressões aritméticas

Exemplo resolvido

Gramática de expressões aritméticas (não-terminal E , terminais `num`, `+`, `*`, `(`, `)`):

$$E \rightarrow E + E \mid E * E \mid \text{num} \mid (E)$$

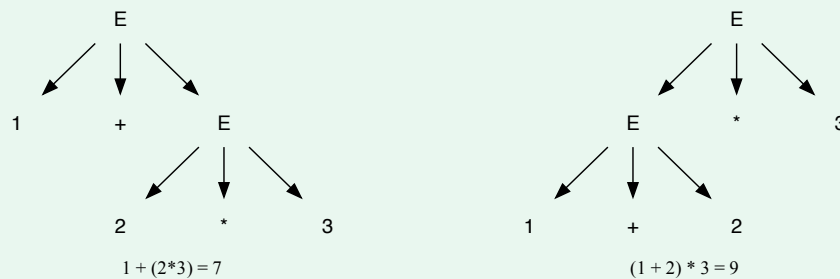
Esta gramática é **ambígua** — a palavra `1+2+3` admite duas árvores:



Ambas as árvores calculam o mesmo resultado (6), mas são **árvores diferentes** — isso já basta para a gramática ser ambígua, mesmo sem nenhuma consequência prática neste caso particular.

Exemplo resolvido

A palavra `1+2*3` já revela o problema **na prática**: as duas árvores possíveis dão **resultados diferentes**.



Se um compilador usasse esta gramática tal como está, a árvore que calhasse ser construída determinaria se `1+2*3` valeria 7 ou 9 — um comportamento imprevisível e obviamente inaceitável para uma linguagem de programação.

Eliminar ambiguidade: associatividade e precedência

Frequentemente consegue-se eliminar a ambiguidade **reescrevendo a gramática** (sem mudar a linguagem que ela descreve). Para expressões aritméticas há duas escolhas a fixar explicitamente:

- **Associatividade** dos operadores: `1+2+3` interpretado à esquerda $((1+2)+3)$ ou à direita $(1+(2+3))$?
- **Prioridade (precedência)** entre operadores: `1+2*3` interpretado como $1 + (2 \times 3)$ ou como $(1+2) \times 3$?

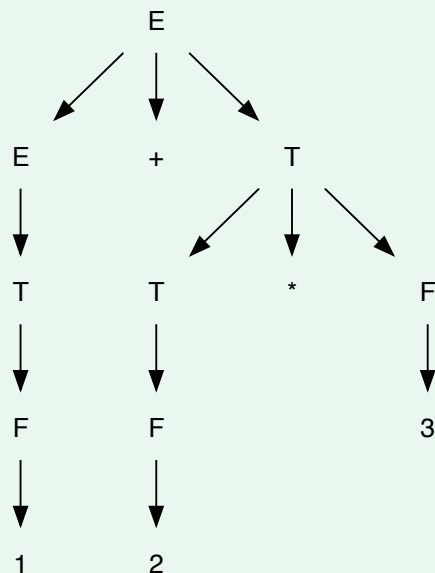
Definição — Gramática de expressões desambiguada

$$E \rightarrow E + T \mid T \quad T \rightarrow T * F \mid F \quad F \rightarrow \text{num} \mid (E)$$

Novos não-terminais e ideia da construção: uma **expressão** (E) é uma soma de **termos** (T); um **termo** é um produto de **fatores** (F); um **fator** é uma constante ou uma expressão entre parêntesis. As produções de E e T têm **recursão à esquerda** ($E \rightarrow E + T$, não $E \rightarrow T + E$) — é isso que força **associatividade à esquerda** para $+$ e $*$. E como só se pode chegar a um $*$ **através** de um T (nunca diretamente a partir de E), um $*$ “agarra” sempre o seu operando antes de ele poder ser usado numa soma — é isso que dá **precedência maior** ao $*$ sobre o $+$.

Exemplo resolvido

Com a gramática desambiguada, $1+2*3$ só tem **uma** árvore possível:



E (raiz) usa $E \rightarrow E + T$: filho esquerdo $E \rightarrow T \rightarrow F \rightarrow 1$; filho direito T , que por sua vez usa $T \rightarrow T * F$: $T \rightarrow F \rightarrow 2$ à esquerda do $*$, $F \rightarrow 3$ à direita. O $*$ fica “preso” dentro do ramo direito do $+$, antes de esse ramo poder combinar-se com o 1 — por isso o resultado é sempre $1 + (2 \times 3) = 7$, nunca $(1 + 2) \times 3$.

Prática agora — Ambiguidade em programas sequenciais (Folha lab. 3, Exercício 2)

Gramática de “programas sequenciais” (também um exercício dos slides): $S \rightarrow S; S \mid \text{ident} = E \mid \text{ident}++$ e $E \rightarrow \text{ident} \mid \text{num} \mid E + E$.

Ex. 2(a) Mostra que é ambígua, com duas árvores diferentes para a mesma frase. O enunciado pergunta se há mais do que um exemplo: há, em **dois** sítios distintos. Um é nas expressões ($E \rightarrow E + E$, o mesmo problema do $E \rightarrow E + E$ acima). O outro é nas próprias instruções: $S \rightarrow S; S$ tem a mesma forma que $E \rightarrow E + E$, por isso tem o mesmo problema de associatividade.

Ex. 2(b) Reescreve a gramática sem ambiguidade. Usa a recursão à esquerda de $E \rightarrow E + T \mid T$ acima, aplicada nos dois sítios (também a S).

Soluções, uma alínea de cada vez: [SOLUCOES_Compiladores.html](#)

O problema do “dangling else”

Muitas linguagens de programação permitem `if/then` com `else` opcional:

$$S \rightarrow \text{if } E \text{ then } S \text{ else } S \mid \text{if } E \text{ then } S \mid \dots$$

Atenção — Dangling else

Esta gramática é ambígua de uma forma particular: em

`if e1 then if e2 then s1 else s2`

o `else s2` pode ligar-se ao `if e1` ou ao `if e2` — duas interpretações válidas segundo a gramática:

- `if e1 then { if e2 then s1 else s2 }` (else liga ao `if` mais próximo)
- `if e1 then { if e2 then s1 } else s2` (else liga ao `if` mais afastado)

Normalmente prefere-se a primeira: **o `else` associa-se sempre ao `if` mais próximo** (a convenção universal na maioria das linguagens).

Frequente em exame

Resolução formal, para o caso de se preferir corrigir a gramática em vez de tratar isto no analisador: introduzir dois não-terminais, M (*matched statements* — instruções onde todo `if` já tem o seu `else`) e U (*unmatched* — instruções com um `if` ainda por fechar):

$$S \rightarrow M \mid U$$

$$M \rightarrow \text{if } E \text{ then } M \text{ else } M \mid \dots$$

$$U \rightarrow \text{if } E \text{ then } S \mid \text{if } E \text{ then } M \text{ else } U$$

A ideia: um `if...then...else...` só pode ser M (fechado) se **ambos** os ramos forem M — isto impede que o `then` interno fique “por fechar” dentro de um `if...else` já completo, forçando o `else` externo a “descer” e agarrar o `if` mais próximo disponível. Na prática, contudo, **é frequente preferir não mexer na gramática** e resolver a ambiguidade diretamente na implementação do analisador sintático: é isso que faz um analisador LR, que escolhe *shift* no conflito que esta gramática cria (ver Aula 5, secção “Conflitos e como resolvê-los”).

Aula 5 — Análise sintática *bottom-up* (LR)

A Aula 4 definiu **o que** o analisador sintático tem de reconhecer (uma gramática independente de contexto) e porque é que a gramática não pode ser ambígua. Esta aula mostra **como** se constrói um analisador que aceita ou rejeita uma sequência de tokens e, pelo caminho, descobre a árvore sintática: a análise **LR**. Plano da aula: o funcionamento de um analisador LR (pilha + tabela), como construir a tabela pelo método **LR(0)**, a melhoria **SLR(1)**, os **conflitos** e como resolvê-los, e, como extra, **LR(1)** e **LALR(1)**.

Análise *bottom-up* e analisadores LR

Definição — Análise LR

LR = *Left-to-right parse, Rightmost derivation*: lê a entrada da esquerda para a direita, um token de cada vez, e constrói uma **derivação mais à direita** (Aula 4, “Derivações”), mas **ao contrário**: parte das folhas (os tokens) e vai juntando pedaços até chegar ao símbolo inicial. Por isso se chama análise ***bottom-up*** (de baixo para cima na árvore), ao contrário da análise *top-down* (LL), que parte do símbolo inicial.

Vantagens da análise LR em relação à LL (*top-down*), segundo os slides:

- reconhece **mais linguagens** (há gramáticas que são LR mas não LL);
- é **mais fácil reescrever** uma gramática para análise LR do que para LL;
- permite **resolver ambiguidades** definindo **prioridades** e **associatividades** dos símbolos, **sem alterar a gramática** (ver “Conflitos e como resolvê-los”, abaixo).

Definição — Analisador LR (autômato de pilha)

Um analisador LR é um **autômato de pilha** com:

- uma **sequência de entrada** (*input*): os terminais ainda por ler;
- uma **pilha** de símbolos (terminais e não-terminais), no início vazia, com a entrada toda por ler.

Em cada passo escolhe **uma** de duas ações:

- **Shift**: tira o próximo terminal da entrada e põe-no no topo da pilha.
- **Reduce**: escolhe uma produção $X \rightarrow \gamma$ tal que os símbolos de γ estão **no topo** da pilha, tira-os e empilha X no lugar deles.

Por razões técnicas, **acrescenta-se à gramática** um símbolo inicial novo S' , um terminal \$ (**marcador de fim** da entrada) e a produção $S' \rightarrow S\$$. Assim o analisador sabe quando a entrada acabou.

Exemplo resolvido — Exemplo 1 dos slides: parêntesis equilibrados

Gramática: $S' \rightarrow S\$$ e $S \rightarrow (S)S \mid \varepsilon$. Entrada $()$ (o analisador vê $()\$$).

Pilha	Entrada	Ação	Porquê
ε	$()\$$	shift	nada no topo para reduzir
$($	$)\$$	reduce $S \rightarrow \varepsilon$	dentro dos parêntesis tem de haver um S (aqui vazio)
$(S$	$)\$$	shift	
(S)	$\$$	reduce $S \rightarrow \varepsilon$	falta o S final de $(S)S$; é vazio
$(S)S$	$\$$	reduce $S \rightarrow (S)S$	o topo é exatamente o lado direito
S	$\$$	accept	sobrou só S e a entrada acabou

Repara que *reduce* $S \rightarrow \varepsilon$ tira **zero** símbolos e empilha um S . A decisão de quando fazer isto é o que a tabela de *parsing* (abaixo) vai resolver.

Lendo os *reduce* de baixo para cima obtém-se a derivação: o último *reduce* foi $S \rightarrow (S)S$, o anterior $S \rightarrow \varepsilon$ (o S da direita), e o primeiro $S \rightarrow \varepsilon$ (o S de dentro):

$$S \Rightarrow (S)\underline{S} \Rightarrow (\underline{S}) \Rightarrow ()$$

É uma derivação **mais à direita**: em $(S)S$ substituiu-se primeiro o S da direita.

Exemplo resolvido — Exemplo 2 dos slides: expressões simples

Gramática: $E' \rightarrow E\$$ e $E \rightarrow E + n \mid n$. Entrada $n+n$.

Pilha	Entrada	Ação
ε	$n+n\$$	shift
n	$+n\$$	reduce $E \rightarrow n$
E	$+n\$$	shift
$E+$	$n\$$	shift
$E+n$	$\$$	reduce $E \rightarrow E + n$
E	$\$$	accept

Os *reduce* lidos de baixo para cima: primeiro $E \rightarrow E + n$, depois $E \rightarrow n$. Dá a derivação mais à direita $E \Rightarrow E + n \Rightarrow n + n$.

No passo 4 (pilha $E+$, entrada $n\$$) repara que **não** se reduz o n que acabou de entrar para E : isso daria $E + E$, que não é lado direito de nenhuma produção. O analisador tem de saber **o que está por baixo** do topo da pilha, não só o topo. É isso que motiva os estados.

Tabela de *parsing* LR e o algoritmo

Como escolhe o autómato a próxima ação? Olha para

- a **configuração da pilha** (não só o símbolo do topo, como se viu no Exemplo 2);
- e possivelmente para os **próximos símbolos da entrada** (*look-ahead*).

Para não ter de analisar a pilha inteira em cada passo, **resume-se a configuração da pilha num número inteiro, o estado**. O autómato muda de estado sempre que empilha ou desempilha, e as ações estão escritas numa **tabela de *parsing* LR**.

Definição — Tabela de parsing LR

- Cada **linha** é um **estado** (um inteiro).
- Cada **coluna** é um **símbolo** (terminais à esquerda, não-terminais à direita).
- Cada **entrada** tem uma ação:
 - $s\ q$ (*shift q*): passa o próximo terminal para a pilha e vai para o estado q ;
 - $r\ k$ (*reduce k*), sendo $X \rightarrow \alpha_1 \dots \alpha_n$ a produção número k : (1) tira da pilha os n símbolos do lado direito; (2) fica no estado que estava por baixo deles, e põe X no topo; (3) procura na tabela a entrada “ $g\ q$ ” na coluna X desse estado; (4) vai para o estado q ;
 - $g\ q$ (*go q*, ou *goto*): muda para o estado q (só aparece nas colunas dos não-terminais, e só se usa depois de um *reduce*);
 - a (*accept*): termina e aceita a sequência;
 - **vazia**: erro de sintaxe.

Exemplo resolvido — Tabela dos slides

Gramática (com as produções numeradas):

Nº	Produção
(0)	$T' \rightarrow T \$$
(1)	$T \rightarrow R$
(2)	$T \rightarrow aTc$
(3)	$R \rightarrow \varepsilon$
(4)	$R \rightarrow bR$

	a	b	c	\$	T	R
0	s3	s4	r3	r3	g1	g2
1				a		
2			r1	r1		
3	s3	s4	r3	r3	g5	g2
4		s4	r3	r3		g6
5			s7			
6			r4	r4		
7			r2	r2		

Como se lê: no estado 0 com próximo símbolo **a**, faz *shift* e vai para o estado 3. No estado 3 com próximo **c**, faz *reduce* pela regra 3 ($R \rightarrow \varepsilon$). No estado 0, depois de um *reduce* para T , vai para o estado 1 (g1). No estado 1 com próximo **\$**, aceita.

Exemplo resolvido — Análise de aabbbcc com a tabela acima (1/2)

A coluna “Estados” mostra a **pilha de estados inteira** (o topo é o último número); os slides só mostram o topo. A coluna “Símbolos” é a pilha de símbolos correspondente (um símbolo por cada estado acima do 0).

Estados	Símbolos	Entrada	Ação
0	ε	aabbbcc\$	(0,a) = s3: shift 3
0 3	a	abbbcc\$	(3,a) = s3: shift 3
0 3 3	a a	bbcc\$	(3,b) = s4: shift 4
0 3 3 4	a a b	bbcc\$	(4,b) = s4: shift 4
0 3 3 4 4	a a b b	bcc\$	(4,b) = s4: shift 4
0 3 3 4 4 4	a a b b b	cc\$	(4,c) = r3
0 3 3 4 4 4 6	a a b b b R	cc\$	(6,c) = r4
0 3 3 4 4 6	a a b b R	cc\$	(6,c) = r4

Os *reduce* explicados um a um:

- (4,c) = **r3**, $R \rightarrow \varepsilon$: tira **0** estados. O topo continua 4; (4,R) = g6, empilha 6.
- (6,c) = **r4**, $R \rightarrow bR$: tira **2** estados (os de b e R), ficando 0 3 3 4 4. Topo 4; (4,R) = g6, empilha 6: 0 3 3 4 4 6.
- (6,c) = **r4** outra vez: tira 2, fica 0 3 3 4; (4,R) = g6, empilha 6.

Exemplo resolvido — Análise de aabbbcc com a tabela acima (2/2)

Estados	Símbolos	Entrada	Ação
0 3 3 4 6	a a b R	cc\$	(6,c) = r4
0 3 3 2	a a R	cc\$	(2,c) = r1
0 3 3 5	a a T	cc\$	(5,c) = s7: shift 7
0 3 3 5 7	a a T c	c\$	(7,c) = r2
0 3 5	a T	c\$	(5,c) = s7: shift 7
0 3 5 7	a T c	\$	(7,\$) = r2
0 1	T	\$	(1,\$) = a: accept

- (6,c) = **r4**, $R \rightarrow bR$: tira 2 de 0 3 3 4 6, fica 0 3 3. Topo 3; (3,R) = g2, empilha 2.
- (2,c) = **r1**, $T \rightarrow R$: tira 1, fica 0 3 3. (3,T) = g5, empilha 5.
- (7,c) = **r2**, $T \rightarrow aTc$: tira **3** (os de a, T, c) de 0 3 3 5 7, fica 0 3. (3,T) = g5, empilha 5.
- (7,\$) = **r2**, $T \rightarrow aTc$: tira 3 de 0 3 5 7, fica 0. (0,T) = g1, empilha 1. Com \$ na entrada, (1,\$) = a: aceita.

Definição — Algoritmo de parsing LR

```

stack = empty; push(0, stack); next = getToken()
loop
  case table[top(stack), next] of
    shift s: push(s, stack); next = getToken()
    reduce p: let X = lado esquerdo da producao p
              n = comprimento do lado direito da producao p
              pop n elementos da pilha
              lookup table[top(stack), X] e encontra "go s"
              push(s, stack)
    accept: termina com sucesso
    vazio: reporta erro

```

Observações dos slides sobre o algoritmo:

- Em cada passo a decisão vem **só da tabela**.
- Basta guardar **os estados** na pilha: cada estado já representa uma configuração de símbolos. Os símbolos não são precisos (mas ajudam a perceber a derivação, por isso aparecem nos exemplos).
- **A parte crucial é construir a tabela**, e isso faz-se **a partir da gramática**, antes e independentemente de qualquer entrada.

Definição — LR(0), LR(1), LR(k)

A tabela pode decidir as ações só com a pilha, ou também com os próximos terminais (*look-ahead*):

- **LR(0)**: só a pilha (0 símbolos de *look-ahead*);
- **LR(1)**: 1 símbolo de *look-ahead*;
- **LR(k)**: k símbolos de *look-ahead* (o caso geral).

A tabela LR(0) é a mais simples de construir, mas **reconhece poucas linguagens**. LR(k) com $k \geq 2$ dá tabelas **muito grandes**. **LR(1) é suficiente para a maior parte das linguagens de programação**.

Autômato e tabela LR(0)

Items LR(0)

Definição — Item LR(0)

Um **item** é uma produção com uma **posição marcada** no lado direito (um ponto, aqui $\bullet$). Os **estados** do autômato LR vão ser **conjuntos de items**.

$A \rightarrow \beta \bullet \gamma$ quer dizer: β **já está no topo da pilha**, e o autômato pode continuar reconhecendo γ .

- $A \rightarrow \bullet \gamma$ é um **item inicial**: ainda não se reconheceu nada; podemos começar com γ .
- $A \rightarrow \gamma \bullet$ é um **item completo**: γ está todo no topo da pilha e podemos reconhecer A (**reduce**).

Exemplo resolvido

A gramática dos parêntesis tem 3 produções e **9 items** (uma produção com lado direito de comprimento n dá $n + 1$ items; $S \rightarrow \varepsilon$ dá só um, $S \rightarrow \bullet$):

Produções	Items
$S' \rightarrow S \$$	$S' \rightarrow \bullet S \$$ $S' \rightarrow S \bullet \$$ $S' \rightarrow S \$ \bullet$
$S \rightarrow (S) S$	$S \rightarrow \bullet (S) S$ $S \rightarrow (\bullet S) S$ $S \rightarrow (S \bullet) S$
	$S \rightarrow (S) \bullet S$ $S \rightarrow (S) S \bullet$
$S \rightarrow \varepsilon$	$S \rightarrow \bullet$

Por exemplo, $S \rightarrow (S) \bullet S$: no topo da pilha está (S) e o autômato já reconheceu (S) ; pode continuar com um S .

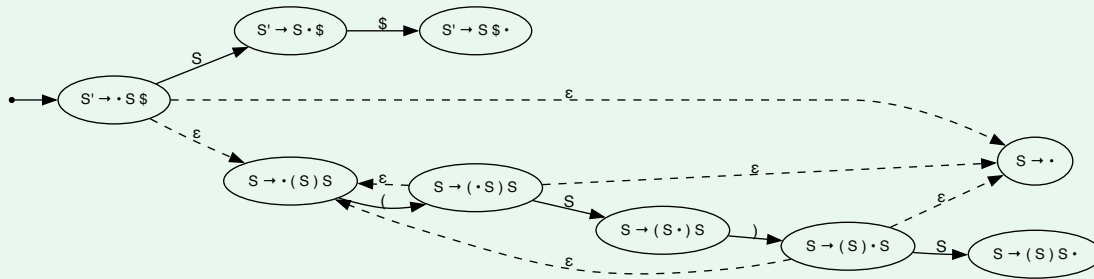
Do NFA de items ao DFA

Definição — Transições entre items

Primeiro constrói-se um **NFA** cujos estados são os items:

- **Transição por um símbolo X** (terminal ou não-terminal): de $A \rightarrow \alpha \bullet X \gamma$ para $A \rightarrow \alpha X \bullet \gamma$ (o ponto “salta” o X). Por um **terminal** acontece depois de um **shift**; por um **não-terminal** acontece depois de um **reduce** (é o *go*).
- **Transições- ϵ** : sempre que o ponto está antes de um **não-terminal** B , em $A \rightarrow \alpha \bullet B \gamma$, acrescenta-se uma transição- ϵ para **todos** os items iniciais de B , $B \rightarrow \bullet \beta$. Ideia: para avançar sobre B primeiro é preciso reconhecer um B , começando uma das suas produções.
- **Estado inicial**: o item $S' \rightarrow \bullet S \$$.
- **Não há estados finais**. A aceitação acontece quando se faria *shift* do $\$$; na tabela é a ação *accept*.

Exemplo resolvido — NFA de items da gramática dos parêntesis



Setas a tracejado: transições- ϵ . Por exemplo, de $S \rightarrow (\bullet S)S$ saem: a transição por S para $S \rightarrow (S\bullet)S$, e duas transições- ϵ para os items iniciais de S ($S \rightarrow \bullet(S)S$ e $S \rightarrow \bullet$), porque o ponto está antes do não-terminal S .

Por causa das transições- ϵ este autômato é **não determinístico**. Converte-se num **DFA** com a **construção de subconjuntos** (Aula 2): os estados do DFA são **conjuntos de items**. Na prática não é preciso desenhar o NFA; faz-se diretamente com duas operações:

Definição — Fecho e transição (construção direta do DFA)

- **fecho(I)** (é o ϵ -closure da Aula 2): começa com os items de I ; para cada item com o ponto antes de um não-terminal B , acrescenta **todos** os items $B \rightarrow \bullet \beta$; repete até não entrar nada novo.
- **goto(I, X)**: pega nos items de I com o ponto antes de X , avança o ponto sobre X , e aplica o **fecho** ao resultado.

Estado inicial: fecho($\{S' \rightarrow \bullet S \$\}$). Depois calcula-se goto(I, X) para cada estado I já encontrado e cada símbolo X ; cada conjunto novo é um estado novo. Pára quando não aparecem conjuntos novos.

Exemplo resolvido — DFA LR(0) dos parêntesis, passo a passo (1/2)

Gramática: (0) $S' \rightarrow S \$$, (1) $S \rightarrow (S)S$, (2) $S \rightarrow \epsilon$. Numeração dos estados como nos slides.

Estado 0 = fecho($\{S' \rightarrow \bullet S \$\}$). O ponto está antes de S , por isso entram os itens iniciais de S : $S \rightarrow \bullet(S)S$ e $S \rightarrow \bullet$. Nestes dois, o ponto está antes de $($ (terminal) ou no fim: não entra mais nada.

$$0 = \{S' \rightarrow \bullet S \$, S \rightarrow \bullet(S)S, S \rightarrow \bullet\}$$

Transições de 0 (os símbolos que aparecem logo a seguir a um ponto são $($ e S ; com $)$ e $\$$ não há transição):

- goto(0, $($): só $S \rightarrow \bullet(S)S$ tem o ponto antes de $($; avança para $S \rightarrow (\bullet S)S$. Fecho: ponto antes de S , entram $S \rightarrow \bullet(S)S$ e $S \rightarrow \bullet$. Conjunto novo: **estado 2** = $\{S \rightarrow (\bullet S)S, S \rightarrow \bullet(S)S, S \rightarrow \bullet\}$.
- goto(0, S): só $S' \rightarrow \bullet S \$$; avança para $S' \rightarrow S \bullet \$$. Ponto antes de $\$$ (terminal): o fecho não acrescenta nada. **Estado 1** = $\{S' \rightarrow S \bullet \$\}$.

Transições de 1: só por $\$$, que é o *accept* (não se cria estado).

Exemplo resolvido — DFA LR(0) dos parêntesis, passo a passo (2/2)

Transições de 2 = $\{S \rightarrow (\bullet S)S, S \rightarrow \bullet(S)S, S \rightarrow \bullet\}$:

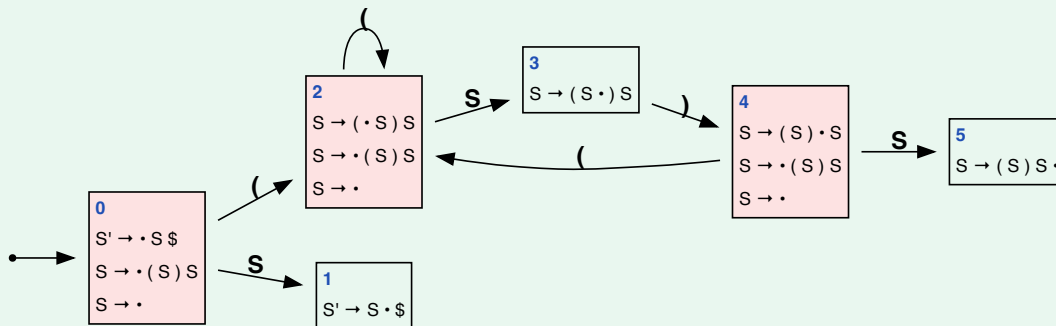
- goto(2, $($): de $S \rightarrow \bullet(S)S$ vem $S \rightarrow (\bullet S)S$, e o fecho dá exatamente o **estado 2** outra vez (um ciclo em 2).
- goto(2, S): de $S \rightarrow (\bullet S)S$ vem $S \rightarrow (S \bullet)S$. Ponto antes de $)$: fecho não acrescenta nada. **Estado 3** = $\{S \rightarrow (S \bullet)S\}$.

Transições de 3: goto(3, $)$ dá $S \rightarrow (S) \bullet S$. Ponto antes de S : entram $S \rightarrow \bullet(S)S$ e $S \rightarrow \bullet$. **Estado 4** = $\{S \rightarrow (S) \bullet S, S \rightarrow \bullet(S)S, S \rightarrow \bullet\}$.

Transições de 4:

- goto(4, $($): de $S \rightarrow \bullet(S)S$ vem o **estado 2** (mesmo conjunto que antes).
- goto(4, S): de $S \rightarrow (S) \bullet S$ vem $S \rightarrow (S)S \bullet$. **Estado 5** = $\{S \rightarrow (S)S \bullet\}$.

Transições de 5: nenhuma (o único item é completo). Não apareceram conjuntos novos: o DFA tem os estados 0 a 5.



A rosa: os estados que vão ter conflitos na tabela LR(0) (ver a seguir).

Construção da tabela LR(0)

Definição — Tabela LR(0)

Numeram-se os estados do DFA (as linhas) e preenche-se:

1. Cada transição por um **terminal** t , de i para j : $s\ j$ na entrada (i, t) .
2. Cada transição por um **não-terminal** X , de i para j : $g\ j$ na entrada (i, X) .
3. Cada estado i com um **item completo** $A \rightarrow \gamma \bullet$ (produção número k): $r\ k$ em **todas** as colunas de terminais da linha i (LR(0) não olha para o próximo símbolo).
4. No estado que contém $S' \rightarrow S \bullet \$$, na coluna $\$$: **accept**.

A gramática é **LR(0)** se **cada entrada tiver no máximo uma ação**. Duas ações na mesma entrada são um **conflito**.

Exemplo resolvido — Tabela LR(0) dos parêntesis

Aplicando as quatro regras ao DFA acima:

1. *Shifts*: $0 \xrightarrow{(} 2$, $2 \xrightarrow{(} 2$, $3 \xrightarrow{)} 4$, $4 \xrightarrow{(} 2$: s2 em $(0, ($), $(2, ($), $(4, ($), e s4 em $(3,)$).
2. *Gotos*: $0 \xrightarrow{S} 1$, $2 \xrightarrow{S} 3$, $4 \xrightarrow{S} 5$: g1, g3, g5 na coluna S .
3. *Reduces*: os estados 0, 2 e 4 têm o item completo $S \rightarrow \bullet$ (produção 2): r2 em **todas** as colunas terminais dessas linhas. O estado 5 tem $S \rightarrow (S)S \bullet$ (produção 1): r1 em todas as colunas.
4. *Accept*: estado 1, coluna $\$$.

	(	)	\$	S
0	s2, r2	r2	r2	g1
1			a	
2	s2, r2	r2	r2	g3
3		s4		
4	s2, r2	r2	r2	g5
5	r1	r1	r1	

Nos estados 0, 2 e 4, com próximo símbolo $($, há **duas ações**: *shift* (começar um par de parêntesis novo) ou *reduce* $S \rightarrow \varepsilon$ (o S ali é vazio). São **conflitos shift/reduce**: **a gramática dos parêntesis não é LR(0)**. (Resolve-se com SLR(1), na secção seguinte.)

Exemplo resolvido — Exercício 3(a) (Folha lab. 3) = Exercício 1 dos slides: construção

Mostrar que $A \rightarrow (A) \mid a$ é LR(0). Gramática aumentada: (0) $A' \rightarrow A \$$, (1) $A \rightarrow (A)$, (2) $A \rightarrow a$.

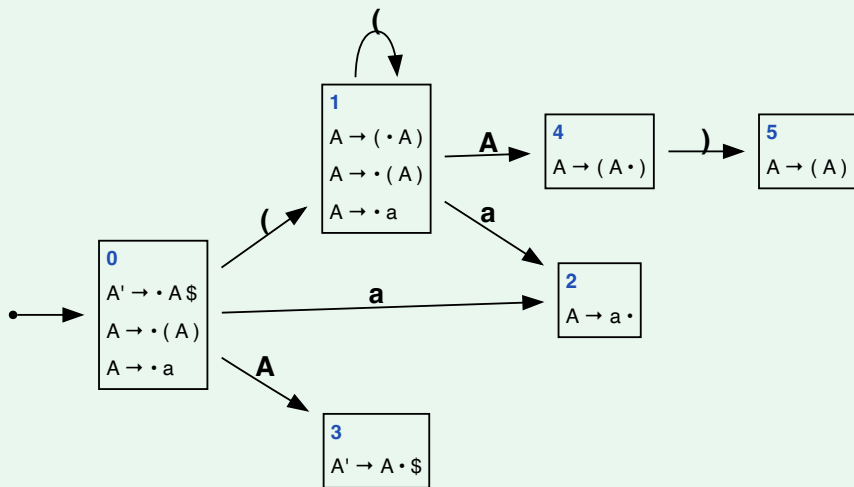
Estado 0 = fecho($\{A' \rightarrow \bullet A \$\}$): ponto antes de A , entram $A \rightarrow \bullet(A)$ e $A \rightarrow \bullet a$ (ponto antes de terminais: pára). $0 = \{A' \rightarrow \bullet A \$, A \rightarrow \bullet(A), A \rightarrow \bullet a\}$.

- goto(0, $($) = fecho($\{A \rightarrow (\bullet A)\}$) = $\{A \rightarrow (\bullet A), A \rightarrow \bullet(A), A \rightarrow \bullet a\}$: **estado 1**.
- goto(0, a) = $\{A \rightarrow a\bullet\}$: **estado 2**.
- goto(0, A) = $\{A' \rightarrow A\bullet \$\}$: **estado 3**.

Transições de 1: goto(1, $($) = fecho($\{A \rightarrow (\bullet A)\}$) = **estado 1** outra vez; goto(1, a) = $\{A \rightarrow a\bullet\}$ = **estado 2**; goto(1, A) = $\{A \rightarrow (A\bullet)\}$: **estado 4**.

Transições de 2 (só item completo) e **de 3** (só $\$, \text{accept}$): nenhuma.

Transições de 4: goto(4, $)$) = $\{A \rightarrow (A)\bullet\}$: **estado 5**, sem transições.



Exemplo resolvido — Exercício 3(a) (Folha lab. 3): tabela e verificação

Shifts: s1 em (0,() e (1,(); s2 em (0,a) e (1,a); s5 em (4,)). *Gotos*: g3 em (0,A), g4 em (1,A). *Reduces*: estado 2 ($A \rightarrow a\bullet$, produção 2) r2 em todas as colunas terminais; estado 5 ($A \rightarrow (A)\bullet$, produção 1) r1 em todas. *Accept*: (3,\$).

		(	)	a	\$	A
0	s1			s2		g3
1	s1			s2		g4
2	r2	r2	r2	r2	r2	
3					a	
4				s5		
5	r1	r1	r1	r1	r1	

Nenhuma entrada tem duas ações: os estados com *reduce* (2 e 5) não têm nenhum *shift*, e vice-versa. Logo a gramática é **LR(0)**.

Teste com ((a)):

Estados	Símbolos	Entrada	Ação
0	ε	((a))\$	s1
0 1	(	(a))\$	s1
0 1 1	((	a))\$	s2
0 1 1 2	((a	))\$	r2: tira 1, topo 1, (1,A) = g4
0 1 1 4	((A	))\$	s5
0 1 1 4 5	((A)	)\$	r1: tira 3, topo 1, (1,A) = g4
0 1 4	(A	)\$	s5
0 1 4 5	(A)	\$	r1: tira 3, topo 0, (0,A) = g3
0 3	A	\$	accept

Prática agora — Autômato e tabela LR(0) (Folha lab. 3, Exercícios 4 e 5)

Ex. 4 Gramática $E \rightarrow (L) \mid a, L \rightarrow L, E \mid E$.

- (a) Derivação de ((a),a,(a,a)). Usa as derivações da Aula 4; escolhe sempre o não-terminal mais à esquerda para não te perderes.
- (b) Autômato LR(0). Atenção ao estado depois de (: tem o ponto antes de L , e $L \rightarrow \bullet L, E$ também tem o ponto antes de L .
- (c) É LR(0)? Procura estados com um item completo e outro item (com *shift* ou outro *reduce*).

Ex. 5 Declarações de C: $Decl \rightarrow Type Varlist ;, Type \rightarrow int \mid float, Varlist \rightarrow Varlist, ident \mid ident$.

- (a) Autômato e tabela LR(0), como no Exercício 3(a) acima.
- (b) Simula `int x,y,z;` com a pilha de estados, como no teste de ((a)) acima (o analisador lexical dá `int ident, ident, ident;`).

Soluções, uma alínea de cada vez: SOLUCOES_Compiladores.html

Análise SLR(1)

O autômato LR(0) usa **só a pilha** para decidir quando fazer *reduce*. Isso gera conflitos com frequência: **muitas gramáticas úteis não são LR(0)** (a dos parêntesis, acima, já não era). Reconhecem-se muito mais linguagens se também se usar o **próximo terminal** (*look-ahead*). A extensão mais simples é a **SLR(1)** (*Simple LR, 1 symbol look-ahead*), que precisa do conjunto FOLLOW.

FIRST e FOLLOW**Atenção — Nota adicional: FIRST e FOLLOW**

Nota adicional (não estava explícito nos slides: usam FOLLOW sem o definir). São precisas três noções, calculadas a partir da gramática.

- **Anulável:** X é anulável se $X \Rightarrow^* \varepsilon$ (por exemplo, se existe $X \rightarrow \varepsilon$, ou $X \rightarrow YZ$ com Y e Z anuláveis).
- **FIRST(α):** os terminais que podem aparecer **no início** de uma palavra derivada de α . Para $\alpha = X_1X_2 \dots X_n$: junta FIRST(X_1); se X_1 for anulável, junta também FIRST(X_2); e assim por diante, enquanto os anteriores forem anuláveis. O FIRST de um terminal t é $\{t\}$.
- **FOLLOW(A):** os terminais que podem aparecer **imediatamente a seguir** a A numa forma derivada de S' . Como $S' \rightarrow S\$$, o $\$$ está sempre em FOLLOW(S).

Cálculo de FOLLOW. Começa com todos vazios e percorre cada ocorrência de um não-terminal A no lado direito de cada produção, $X \rightarrow \alpha A \beta$:

1. junta FIRST(β) a FOLLOW(A) (o que vem depois de A);
2. se β é vazio ou anulável, junta FOLLOW(X) a FOLLOW(A) (o que vem depois de X também pode vir depois de A).

Repete a passagem até nenhum conjunto mudar.

Exemplo resolvido — FOLLOW da gramática dos parêntesis

Produções: $S' \rightarrow S\$$, $S \rightarrow (S)S$, $S \rightarrow \varepsilon$. As ocorrências de S em lados direitos são três:

1. $S' \rightarrow \underline{S}\$$: depois de S vem $\$$. FIRST($\$$) = $\{\$, \}$, logo $\$ \in \text{FOLLOW}(S)$.
2. $S \rightarrow (\underline{S})S$: depois vem $)S$. FIRST($)S$) = $\{)\}$ (começa por um terminal), logo $) \in \text{FOLLOW}(S)$.
3. $S \rightarrow (S)\underline{S}$: depois não vem nada, por isso junta FOLLOW(S) a FOLLOW(S): não acrescenta nada.

Segunda passagem: nada muda. FOLLOW(S) = $\{), \$\}$. Faz sentido: um S acaba sempre antes de um $)$ (dentro de parêntesis) ou no fim da entrada.

Exemplo resolvido — FOLLOW das expressões aritméticas (Aula 4)

Gramática desambiguada da Aula 4 com $E' \rightarrow E\$$: $E \rightarrow E + T \mid T$, $T \rightarrow T * F \mid F$, $F \rightarrow \text{num} \mid (E)$. Nada é anulável, por isso FIRST(F) = FIRST(T) = FIRST(E) = $\{\text{num}, (\}$.

- E aparece em $E' \rightarrow \underline{E}\$$ (dá $\$$), em $E \rightarrow \underline{E} + T$ (dá $+$) e em $F \rightarrow (\underline{E})$ (dá $)$). FOLLOW(E) = $\{\$, +,)\}$.
- T aparece em $E \rightarrow E + \underline{T}$ e $E \rightarrow \underline{T}$ (no fim: junta FOLLOW(E)) e em $T \rightarrow \underline{T} * F$ (dá $*$). FOLLOW(T) = $\{\$, +,), *\}$.
- F aparece em $T \rightarrow T * \underline{F}$ e $T \rightarrow \underline{F}$, sempre no fim: junta FOLLOW(T). FOLLOW(F) = $\{\$, +,), *\}$.

Segunda passagem: nada muda.

Tabela SLR(1)**Definição — Tabela SLR(1)**

O algoritmo de *parsing* é o mesmo, e o autômato é o mesmo do LR(0). Só muda a construção da tabela:

- *shift* e *goto* colocam-se como antes;
- um *reduce* por um item completo $A \rightarrow \gamma \bullet$ coloca-se **apenas nas colunas dos terminais que estão em FOLLOW(A)** (em vez de em todas).

Ideia: só faz sentido reduzir para A se o próximo símbolo puder aparecer a seguir a um A . A gramática é **SLR(1)** se esta tabela não tiver conflitos.

Exemplo resolvido — Tabela SLR(1) dos parêntesis

Mesmo DFA de antes (estados 0 a 5). Shifts e gotos iguais. Os *reduce* agora só vão para as colunas de $\text{FOLLOW}(S) = \{ \text{)}, \$ \}$:

- estados 0, 2 e 4 ($S \rightarrow \bullet$, produção 2): r2 nas colunas $\text{)} e \$$, **não** na coluna (;
- estado 5 ($S \rightarrow (S)\bullet$, produção 1): r1 nas colunas $\text{)} e \$$.

		(	)	\$	S
0	s2	r2	r2	g1	
1			a		
2	s2	r2	r2	g3	
3		s4			
4	s2	r2	r2	g5	
5		r1	r1		

Já **não há conflitos**: a gramática dos parêntesis é **SLR(1)** (apesar de não ser LR(0)). O conflito na coluna (desapareceu porque (não está em $\text{FOLLOW}(S)$: se o próximo símbolo é (, um S vazio nunca estaria certo ali, e o analisador faz *shift*.

Exemplo resolvido — Análise de $(())$ com a tabela SLR(1)

Estados	Símbolos	Entrada	Ação
0	ε	$(())\$$	s2
0 2	$($	$(())\$$	s2
0 2 2	$(($	$(())\$$	r2: tira 0, topo 2, $(2,S) = g3$
0 2 2 3	$((S$	$(())\$$	s4
0 2 2 3 4	$((S)$	$(())\$$	r2: tira 0, topo 4, $(4,S) = g5$
0 2 2 3 4 5	$((S) S$	$(())\$$	r1: tira 4, topo 2, $(2,S) = g3$
0 2 3	$(S$	$(())\$$	s4
0 2 3 4	(S)	$(())\$$	r2: tira 0, topo 4, $(4,S) = g5$
0 2 3 4 5	$(S) S$	$(())\$$	r1: tira 4, topo 0, $(0,S) = g1$
0 1	S	$(())\$$	accept

Repara no terceiro passo: estado 2 com) . Em LR(0) a entrada $(2,\text{)})$ também era r2, mas no primeiro passo, com (, havia conflito; aqui a tabela SLR(1) diz s2 sem hesitar.

Pratica agora — SLR(1) (Folha lab. 3, Exercício 3(b)) = Exercício 2 dos slides

Ex. 3(b) Mostra que $T \rightarrow R$, $T \rightarrow aTc$, $R \rightarrow \varepsilon$, $R \rightarrow bR$ é SLR(1): constrói o autômato e a tabela. Justifica que **não** é LR(0) mostrando os conflitos da tabela LR(0).

Pista: é a gramática da tabela dos slides (secção “Tabela de *parsing* LR”), por isso podes comparar no fim. Os slides avisam que a tua numeração dos estados pode sair diferente, mas sem conflitos. Calcula $\text{FOLLOW}(T)$ e $\text{FOLLOW}(R)$ primeiro (a produção $T \rightarrow R$ faz com que $\text{FOLLOW}(T)$ entre em $\text{FOLLOW}(R)$).

Soluções, uma alínea de cada vez: SOLUCOES_Compiladores.html

Conflitos e como resolvê-los

Definição — Conflito

Há um **conflito** quando uma entrada da tabela de *parsing* tem **duas (ou mais) ações**: o autômato tem mais do que uma ação possível.

- **Conflito shift/reduce**: um *shift* e um (ou mais) *reduce* no mesmo estado, para o mesmo símbolo.
- **Conflito reduce/reduce**: dois (ou mais) *reduce* no mesmo estado, para o mesmo símbolo.

Os conflitos **podem** resultar de ambiguidade na gramática. **Mas nem sempre**: a gramática dos parêntesis **não é ambígua** e tinha conflitos shift/reduce na tabela LR(0). Uma gramática ambígua, pelo contrário, dá **sempre** conflitos em qualquer tabela LR (duas árvores quer dizer que, em algum passo, há duas ações que levam a uma aceitação).

Os **geradores de analisadores LR** (Bison para C, Happy para Haskell, próxima aula) avisam de todos os conflitos da gramática. Formas de os eliminar:

- **reescrever a gramática** (se for ambígua; técnicas da Aula 4);
- definir **associatividades e precedências** para os *tokens*;
- escolher **shift em vez de reduce** (é o que os geradores fazem por omissão).

Exemplo resolvido — Dangling else num analisador SLR(1)

Gramática (a ambiguidade foi explicada na Aula 4, “O problema do dangling else”): $S \rightarrow \text{if cond then } S \text{ else } S \mid \text{if cond then } S \mid \text{skip}$.

Ao construir o autômato SLR(1) aparece um estado com estes dois items:

$$S \rightarrow \text{if cond then } S \bullet \qquad S \rightarrow \text{if cond then } S \bullet \text{ else } S$$

Se o próximo *token* é **else**, o analisador pode:

- fazer **shift** do **else**, pelo segundo item (a produção com **else**);
- fazer **reduce** pela produção sem **else**, pelo primeiro item, porque $\text{else} \in \text{FOLLOW}(S) = \{\text{else}, \$\}$ (em $S \rightarrow \text{if cond then } \underline{S} \text{ else } S$ o S é seguido de **else**).

Conflito shift/reduce. Em $\text{if cond then if cond then skip else skip}$, este estado aparece quando a pilha tem $\text{if cond then if cond then } S$ e o próximo é **else**:

- **shift**: o **else** junta-se ao **if de dentro** (o que está no topo da pilha). Dá $\text{if cond then } \{ \text{if cond then skip else skip} \}$.
- **reduce**: fecha o **if de dentro** sem **else**, e o **else** fica para o **if de fora**. Dá $\text{if cond then } \{ \text{if cond then skip} \} \text{ else skip}$.

Optando por shift obtém-se a interpretação usual das linguagens de programação (Pascal, C, Java, ...): o **else** liga-se ao **if** mais próximo. É por isso que “escolher *shift* por omissão” resolve este caso sem mexer na gramática.

Extras: LR(1) e LALR(1)

Apesar de SLR(1) reconhecer mais do que LR(0), **não chega para algumas construções das linguagens de programação**. A análise **LR(1)** é uma generalização de SLR(1) que resolve essas limitações. Contudo, o método LR(1) pode produzir autômatos com **muito mais estados** do que o SLR(1). Na prática, os geradores de analisadores usam uma variante mais “compacta”, a **LALR(1)** (*Look-Ahead LR(1)*). As diferenças entre SLR(1), LR(1) e LALR(1) são bastante técnicas; segundo os slides, **percebendo SLR(1) já se consegue compreender e usar os geradores de analisadores**.

Definição — Items e autômato LR(1)

Os estados LR(0) só representam posições nos lados direitos. Em LR(1) o *look-ahead* entra na **própria definição dos estados**, para os distinguir e evitar conflitos.

- Um **item LR(1)** é um par $(A \rightarrow \alpha \bullet \beta, a)$: um item LR(0) e um terminal a (*look-ahead*). Quer dizer: α está no topo da pilha e o resto da entrada é derivável a partir de βa .
- Os estados continuam a ser **conjuntos** de items.
- **Transição por um símbolo X** : de $(A \rightarrow \alpha \bullet X \gamma, a)$ para $(A \rightarrow \alpha X \bullet \gamma, a)$ (o *look-ahead* não muda).
- **Transição- ε** : de $(A \rightarrow \alpha \bullet B \gamma, a)$ para $(B \rightarrow \bullet \beta, b)$, para **todas** as produções $B \rightarrow \beta$ e **todos** os $b \in \text{FIRST}(\gamma a)$.
- **Tabela**: *shift* e *go* como em LR(0)/SLR(1); *reduce* $A \rightarrow \alpha$ quando o estado tem o item completo $(A \rightarrow \alpha \bullet, a)$ e o **próximo terminal é a** .

Exemplo resolvido — Nota adicional: o início do autômato LR(1) dos parêntesis

Nota adicional (exemplo construído para este resumo; os slides não trazem nenhum). Gramática $S' \rightarrow S \$, S \rightarrow (S) S \mid \varepsilon$.

Estado inicial. Parte de $S' \rightarrow \bullet S \$$ (o *look-ahead* deste item não interessa, porque o $\$$ já está na produção). O ponto está antes de S , com $\gamma = \$$: $\text{FIRST}(\$) = \{\$\}$. Entram $(S \rightarrow \bullet (S) S, \$)$ e $(S \rightarrow \bullet, \$)$.

Depois de $($. De $(S \rightarrow \bullet (S) S, \$)$ vem $(S \rightarrow (\bullet S) S, \$)$. O ponto está antes de S com $\gamma =)S$ e $a = \$$: $\text{FIRST}()S\$) = \{ \}$. Entram $(S \rightarrow \bullet (S) S,))$ e $(S \rightarrow \bullet,))$.

Neste estado o *reduce* $S \rightarrow \varepsilon$ faz-se **só com $)$** (em SLR(1) era com $)$ e $\$$). E um segundo $($ já não volta ao mesmo estado: vai para um estado com $(S \rightarrow (\bullet S) S,))$, que tem outro *look-ahead*. É assim que LR(1) distingue mais situações, e também porque tem mais estados. A LALR(1) junta outra vez os estados LR(1) que só diferem nos *look-aheads* (mesmos items LR(0)), ficando com o número de estados do LR(0), mas com *look-aheads* mais precisos do que o FOLLOW.